

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

Учреждение образования
«Гродненский государственный университет имени Янки Купалы»

Факультет математики и информатики
Кафедра
системного программирования и компьютерной безопасности

КИСЕЛЕВСКИЙ ДАНИИЛ АНДРЕЕВИЧ

**Разработка прототипа семантического поиска вакансий с
использованием языковой модели и векторных представлений**

Курсовая работа по учебной дисциплине
«Модели решения задач в интеллектуальных системах»
студента 3 курса специальности
6-05-0611-03 «Искусственный интеллект»

Научный руководитель
Вувуникян Юрий Микиртычевич,
профессор кафедры
фундаментальной и прикладной
математики,
доктор физико-математических
наук,
профессор

Гродно 2026

РЕЗЮМЕ

Курсовая работа: 83 с., 14 рис., 3 табл., 2 прил., 19 источников.

ИНТЕЛЛЕКТУАЛЬНАЯ СИСТЕМА, СЕМАНТИЧЕСКИЙ ПОИСК, ВЕКТОРНЫЕ ЭМБЕДДИНГИ, SENTENCE-BERT, RAG, БОЛЬШАЯ ЯЗЫКОВАЯ МОДЕЛЬ, PGVECTOR, FASTAPI, STREAMLIT, РАНЖИРОВАНИЕ ВАКАНСИЙ.

Объект исследования: процессы информационного поиска и ранжирования в интеллектуальных системах при поиске вакансий соискателями Республики Беларусь.

Предмет исследования: модели и методы семантического поиска вакансий на основе векторных эмбеддингов и Retrieval-Augmented Generation.

Цель работы: исследовать модели семантического поиска и разработать программное средство поиска вакансий для рынка труда Республики Беларусь.

Методы исследования: системный анализ, векторное моделирование, применение предобученных моделей машинного обучения, программная инженерия, количественная оценка качества поиска.

Исследования и разработки: реализована система поиска вакансий на PostgreSQL с pgvector, автоматическим сбором данных, взвешенной векторизацией, LLM-извлечением фильтров, релаксацией и доменным реранжированием.

Элементы научной новизны: предложен каскад моделей с усилением структурированных полей при векторизации, извлечением взвешенных фильтров, релаксацией по минимальному весу и доменным реранжированием.

Область возможного практического применения: карьерные центры ВУЗов, HR-сервисы, агрегаторы вакансий, учебные и исследовательские проекты по интеллектуальному поиску.

Автор работы подтверждает, что приведённый в курсовой работе расчётно-аналитический материал правильно и объективно отражает состояние исследуемого процесса, а все заимствованные из литературных и других источников теоретические, методологические и методические положения и концепции сопровождаются ссылками на их авторов.

SUMMARY

Course project: 83 p., 14 fig., 3 tabl., 2 app., 19 ref.

INTELLIGENT SYSTEM, SEMANTIC SEARCH, VECTOR EMBEDDINGS, SENTENCE-BERT, RAG, LARGE LANGUAGE MODEL, PGVECTOR, FASTAPI, STREAMLIT, VACANCY RANKING.

The object of the research: information retrieval and ranking processes in intelligent systems for vacancy search by university graduates in the Republic of Belarus.

The subject of the research: models and methods of semantic vacancy search based on vector embeddings and Retrieval-Augmented Generation.

The research objective: to investigate semantic search models and develop a software tool for vacancy search adapted to the labor market of the Republic of Belarus.

The methods of research: system analysis, vector-space modeling, application of pretrained machine learning models, software engineering, and quantitative search quality evaluation.

Researches and developments: an intelligent vacancy search system was implemented using PostgreSQL with pgvector, automated data collection, weighted vectorization, LLM-based filter extraction, filter relaxation, and domain reranking.

The elements of scientific novelty: a cascade of models is proposed, combining amplified structured-field vectorization, weighted filter extraction, minimum-weight filter relaxation, and domain reranking.

The field of possible practical use: university career centers, recruitment services, vacancy aggregators, and educational or research projects in intelligent information retrieval.

The author of work confirms that the design and analytical materials of course project reflect the state of the process under research correctly and impartially, and all theoretical, methodological and methodical statements and conceptions borrowed from the literature and other sources are accompanied by references to their authors.

СОДЕРЖАНИЕ

ПЕРЕЧЕНЬ УСЛОВНЫХ ОБОЗНАЧЕНИЙ	6
ВВЕДЕНИЕ	7
1 ТЕОРЕТИЧЕСКИЕ МОДЕЛИ РЕШЕНИЯ ЗАДАЧ ПОИСКА И РАНЖИРОВАНИЯ В ИНТЕЛЛЕКТУАЛЬНЫХ СИСТЕМАХ	9
1.1 Задача поиска и ранжирования как задача интеллектуальной системы...9	
1.2 Эволюция моделей представления текста.....10	
1.3 Метрики близости в векторных пространствах.....11	
1.4 Архитектура Retrieval-Augmented Generation.....12	
1.5 Выводы по главе 1	13
2 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ПОСТАНОВКА ЗАДАЧИ	14
2.1 Особенности рынка труда Республики Беларусь и задачи поиска работы соискателями.....14	
2.2 Обзор существующих решений и их ограничений	15
2.3 Формулировка задачи и требования к системе.....17	
2.4 Обоснование стека технологий и архитектуры	18
2.5 Выводы по главе 2	20
3 ПРОЕКТИРОВАНИЕ И ПРОГРАММНАЯ РЕАЛИЗАЦИЯ КАСКАДА МОДЕЛЕЙ.....	21
3.1 Общая архитектура системы	21
3.2 Модель хранения знаний: схема базы данных и векторное поле	22
3.3 Модель сбора данных: парсер вакансий rabota.by24	
3.4 Модель векторизации вакансии: взвешенное текстовое представление27	
3.5 Модель извлечения структурированных фильтров через LLM	29
3.6 Модель релаксации фильтров.....33	
3.7 Модель гибридного ранжирования с доменным бустом	36
3.8 Реализация клиентского и серверного слоев	38
3.9 Выводы по главе 3	39
4 ЭКСПЕРИМЕНТАЛЬНАЯ ОЦЕНКА И АНАЛИЗ РАБОТЫ СИСТЕМЫ ...	41

4.1 Постановка эксперимента и методика оценки.....	41
4.2 Функциональное тестирование системы в пользовательских сценариях..	42
4.3 Количественная оценка качества поиска.....	45
4.4 Абляционный анализ: сопоставление конфигураций А и В.....	47
4.5 Визуализация процесса принятия решения	49
4.6 Выводы по главе 4	51
5 НАПРАВЛЕНИЯ ДАЛЬНЕЙШЕГО РАЗВИТИЯ	52
5.1 Усиление моделей retrieval и ранжирования.....	52
5.2 Развитие модели интерпретации запроса и генеративного контура.....	52
5.3 Масштабирование корпуса вакансий и программной архитектуры.....	53
5.4 Выводы по главе 5	54
ЗАКЛЮЧЕНИЕ	56
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	58
ПРИЛОЖЕНИЕ А	60
ПРИЛОЖЕНИЕ Б.....	81

ПЕРЕЧЕНЬ УСЛОВНЫХ ОБОЗНАЧЕНИЙ

ANN	Approximate Nearest Neighbors (приближённый поиск ближайших соседей)	MiniLM	Компактная модель-трансформер (Wang et al., 2020)
API	Application Programming Interface (программный интерфейс)	MRR	Mean Reciprocal Rank (средний обратный ранг)
BERT	Bidirectional Encoder Representations from Transformers	nDCG	Normalized Discounted Cumulative Gain
BM25	Best Matching 25 (ранжирующая функция Okapi BM25)	ORM	Object-Relational Mapping
CRUD	Create, Read, Update, Delete	pgvector	Расширение PostgreSQL для работы с векторными типами
DTO	Data Transfer Object (объект передачи данных)	RAG	Retrieval-Augmented Generation
HNSW	Hierarchical Navigable Small World (индекс ANN)	REST	Representational State Transfer
hit@k	Метрика попадания релевантного результата в топ-k	SBERT	Sentence-BERT (Reimers, Gurevych, 2019)
IR	Information Retrieval (информационный поиск)	SPA	Single Page Application
JSON	JavaScript Object Notation	SQL	Structured Query Language
LLM	Large Language Model (большая языковая модель)	TF-IDF	Term Frequency – Inverse Document Frequency
		UI	User Interface (пользовательский интерфейс)
		UUID	Universally Unique Identifier
		ВУЗ	Высшее учебное заведение
		СУБД	Система управления базами данных

ВВЕДЕНИЕ

Современные сервисы поиска работы в основном опираются на точное совпадение ключевых слов и жесткие фильтры, поэтому запросы со сходным смыслом, но разной формулировкой, нередко приводят к неполной выдаче. Для соискателей, в том числе из высших учебных заведений, данная проблема особенно важна, поскольку на начальном этапе трудоустройства пользовательские запросы часто формулируются нестрого, содержат смешение русского и английского профессионального словаря и требуют интерпретации не только текста запроса, но и содержания вакансии.

Практическая значимость работы состоит в разработке программного средства для семантического поиска вакансий по данным портала rabota.by. Результаты работы могут использоваться в деятельности карьерных центров университетов, в учебном процессе при изучении интеллектуальных систем и информационного поиска, а также как основа для дальнейшего развития специализированных HR сервисов.

Теоретическую основу составляют работы А. Васвани, Дж. Девлина, Н. Раймерс, И. Гуревич, Т. Миколова, П. Льюиса и В. Вана, посвященные трансформерным архитектурам, моделям BERT и Sentence-BERT, векторным представлениям текста и подходу Retrieval Augmented Generation. Существенный вклад в развитие методов поиска и ранжирования внесен также в трудах К. Мэннинга, П. Рагхавана, Х. Шютце, Д. Джурафски и Дж. Мартина. Однако решения, адаптированные к задаче поиска вакансий в условиях рынка труда Республики Беларусь, изучены недостаточно полно.

Обоснование выбора темы связано с содержанием учебной дисциплины «Модели решения задач в интеллектуальных системах». Рассматриваемая задача естественным образом раскладывается на несколько взаимосвязанных моделей: сбор данных о вакансиях, их векторное представление, извлечение фильтров из пользовательского запроса, релаксацию ограничений и итоговое ранжирование. Это позволяет исследовать не отдельный алгоритм, а целостный каскад моделей и оценить вклад его компонентов в качество поиска.

Цель работы: исследовать модели решения задачи семантического поиска в интеллектуальных системах и разработать программное средство для поиска вакансий, адаптированное к рынку труда Республики Беларусь.

Для достижения поставленной цели необходимо решить следующие задачи: изучить теоретические модели информационного поиска и ранжирования; проанализировать предметную область рынка труда Республики Беларусь и существующие средства поиска вакансий; обосновать

постановку задачи, требования к системе и выбор программного стека; спроектировать и реализовать каскад моделей хранения, векторизации, извлечения фильтров, релаксации и ранжирования; выполнить количественную оценку качества поиска и определить вклад отдельных компонентов системы; установить направления дальнейшего развития разработанного решения.

Объектом исследования являются процессы информационного поиска и ранжирования в интеллектуальных системах применительно к задаче подбора вакансий. Предмет исследования составляют программные модели и методы семантического поиска вакансий на основе векторных представлений, языковой модели и архитектуры Retrieval Augmented Generation. В работе использованы методы системного анализа предметной области, математического моделирования векторных представлений текста, применения предобученных моделей машинного обучения, программной реализации и экспериментальной количественной оценки качества поиска.

Курсовая работа состоит из введения, пяти глав, заключения, списка использованных источников и приложений. В первой главе рассматриваются теоретические модели поиска и ранжирования. Во второй главе анализируется предметная область и формулируется постановка задачи. В третьей главе описываются проектирование и программная реализация системы. В четвертой главе приводятся результаты экспериментальной оценки. В пятой главе определяются направления дальнейшего развития разработанного решения.

1 ТЕОРЕТИЧЕСКИЕ МОДЕЛИ РЕШЕНИЯ ЗАДАЧ ПОИСКА И РАНЖИРОВАНИЯ В ИНТЕЛЛЕКТУАЛЬНЫХ СИСТЕМАХ

1.1 Задача поиска и ранжирования как задача интеллектуальной системы

Задачи поиска и ранжирования относятся к базовому классу задач информационного поиска, в котором имеется корпус документов, пользовательский запрос и правило упорядочения документов по степени их релевантности запросу [1], [2]. В наиболее общем виде система должна не просто извлечь документы из хранилища, а сформировать такое ранжирование, при котором первые позиции выдачи содержат наиболее содержательно близкие и практически полезные для пользователя объекты.

Для интеллектуальной системы важен не сам факт нахождения документа, а качество принятого решения о его месте в выдаче. Поэтому поиск вакансий может быть формализован как сопоставление множества описаний вакансий и текстового запроса соискателя, после чего каждому документу назначается оценка релевантности, а результаты сортируются по убыванию этой оценки [1]. В таком представлении вакансия выступает документом предметной области, а пользовательский запрос содержит свернутое описание профессиональных намерений, навыков и ограничений кандидата.

Качество работы поисковой модели принято оценивать с помощью метрик полноты и точности, а также ранговых показателей, отражающих полезность верхней части выдачи. Для пользовательских сценариев подбора вакансий особенно значимы метрики $\text{hit}@k$, MRR и nDCG, поскольку соискатель обычно просматривает лишь несколько первых позиций и ожидает увидеть релевантные предложения уже в верхней части списка [1], [2]. Именно поэтому задача ранжирования в системах трудоустройства не сводится к бинарной фильтрации, а требует тонкой оценки относительной близости документов к запросу.

Система является интеллектуальной в том плане, что она объединяет несколько взаимосвязанных моделей: представление текста, вычисление близости, отбор кандидатов и интерпретацию пользовательского намерения. В отличие от обычного лексического поиска, интеллектуальная система должна учитывать синонимию, неполноту формулировок, смешение русского и английского профессионального словаря и неоднозначность ролей на рынке труда Республики Беларусь [2], [9]. Следовательно, задача поиска вакансий

является частным, но практически значимым случаем задачи поиска и ранжирования в интеллектуальных системах.

1.2 Эволюция моделей представления текста

Исторически первые поисковые системы опирались на точное совпадение строк, регулярные выражения и булевы условия. Такие модели просты, интерпретируемы и хорошо подходят для жестко структурированных запросов, однако они плохо работают в ситуациях, когда один и тот же смысл выражается разными словами или когда запрос пользователя сформулирован неполно [1]. Для рынка вакансий это особенно критично, так как формулировки «стажер Python», «junior backend developer» и «начинающий разработчик на Python» могут описывать схожий класс позиций.

Следующим этапом развития стали лексические модели представления текста, прежде всего bag of words, TF IDF и BM25. В этих моделях документ представляется набором терминов с весами, отражающими их частоту в документе и информативность во всем корпусе. Подобные методы существенно повысили качество поиска по сравнению с точным совпадением, однако по своей природе они по-прежнему измеряют главным образом лексическое пересечение, а не смысловую близость [1].

Дальнейшее развитие методов обработки естественного языка связано с переходом к плотным векторным представлениям и нейросетевым моделям. Архитектура Transformer является основой для построения контекстных языковых моделей, способных учитывать зависимость значения слова от его окружения [3], [4], [5]. На этой базе была предложена модель BERT, в которой представления токенов формируются с учетом двустороннего контекста, что позволило существенно продвинуть задачи классификации, извлечения сущностей и семантического анализа текста [3], [5].

Однако классический BERT не оптимизирован для прямого сравнения предложений и документов в крупном корпусе, поскольку качественное попарное сопоставление требует дорогостоящего совместного пропуска пары текстов через модель. Для задач поиска это ограничение было преодолено в модели Sentence-BERT, где используется bi-encoder (би-энкодер) архитектура: каждый текст независимо преобразуется в вектор фиксированной размерности, после чего сравнение выполняется уже в векторном пространстве [10]. Такой подход делает возможной офлайн индексацию корпуса и быстрый онлайн поиск по готовым представлениям.

Практически важным развитием идеи Sentence-BERT стали компактные дистиллированные модели семейства MiniLM. Они сохраняют приемлемое качество семантического представления текста при существенно меньших вычислительных затратах, что особенно важно для прикладных систем, где требуется регулярно векторизовать большой массив документов и обеспечивать быстрый ответ на пользовательский запрос [10]. Для данного проекта принципиально значимо и то, что мультиязычная модель paraphrase-multilingual-MiniLM-L12-v2 формирует векторы размерности 384 и предназначена для сопоставления предложений на разных языках [18].

Таким образом, эволюция моделей представления текста показывает переход от поиска по совпадению символов к поиску по смыслу. Для задачи семантического поиска вакансий наиболее рациональным оказывается класс bi-encoder моделей семейства Sentence-BERT, поскольку он обеспечивает компромисс между качеством семантического сопоставления, возможностью предварительной индексации корпуса и приемлемыми требованиями к вычислительным ресурсам [10], [18].

1.3 Метрики близости в векторных пространствах

После перехода к плотным эмбедингам задача ранжирования сводится к измерению близости между вектором запроса и векторами документов. На практике для этого используются скалярное произведение, евклидово расстояние и косинусная близость. Выбор конкретной метрики влияет на итоговый порядок выдачи, так как разные меры по-разному учитывают длину вектора и направление в пространстве признаков [1], [2].

Косинусная близость измеряет угол между двумя векторами и потому хорошо подходит для сравнения текстовых представлений, когда важнее совпадение смыслового направления, чем абсолютная длина вектора. Если выходы модели нормированы, сравнение по косинусной близости фактически отражает близость семантических профилей текстов и менее чувствительно к вариациям масштаба представления [1], [10]. Для информационного поиска это удобно тем, что документы разной длины могут сравниваться в единой геометрии пространства.

Евклидово расстояние лучше отражает абсолютную удаленность точек, а скалярное произведение зависит как от направления, так и от длины вектора. Поэтому в задачах семантического поиска текстов косинусная мера обычно оказывается более эффективной и устойчивой, особенно когда bi-encoder модель обучалась именно на задаче сопоставления предложений по

смыслу [2], [10]. В этом случае ранжирование по минимальному косинусному расстоянию естественным образом соответствует поиску наиболее близких по смыслу вакансий.

Для прикладной реализации важна и технологическая поддержка выбранной метрики. Расширение pgvector предоставляет хранение векторов внутри PostgreSQL и поддерживает операции поиска по косинусному расстоянию непосредственно на стороне СУБД [13], [14]. Поскольку в рассматриваемом проекте используется мультязычная MiniLM модель с размерностью вектора 384, сочетание vector(384) и косинусной метрики образует согласованную математическую и программную основу для последующего ранжирования вакансий [14], [18].

1.4 Архитектура Retrieval-Augmented Generation

Архитектура Retrieval-Augmented Generation рассматривает языковую модель не как единственный источник ответа, а как компонент системы, работающий совместно с внешней памятью. В такой постановке сначала выполняется поиск релевантных документов, затем найденный контекст передается генеративной модели, которая формирует итоговый ответ, опираясь на retrieved данные [8], [9]. Это превращает поиск и генерацию в единую композиционную модель решения задачи.

Классическая RAG схема включает несколько функциональных блоков. Retriever отвечает за первичный отбор кандидатов из корпуса, reranker уточняет порядок документов в верхней части выдачи, а generator формирует текстовое объяснение, сводку или ответ на вопрос пользователя на основе переданного контекста [2], [8], [9]. Ограничение контекстного окна генеративной модели делает особенно важным качественный отбор документов на ранних этапах, поскольку именно они определяют фактическую основу финального ответа.

Принципиальное достоинство RAG состоит в снижении вероятности галлюцинаций и в возможности работать с актуальными, локальными и предметно специфичными данными. В отличие от изолированной большой языковой модели, система с внешней памятью может опираться на текущий корпус вакансий, а не только на параметры, полученные в ходе предварительного обучения [9]. Благодаря этому генеративный компонент перестает подменять поиск, а начинает использовать результаты поиска как опору для интерпретации и объяснения.

Для систем поиска вакансий архитектура RAG особенно естественна. Внешней памятью здесь выступает база вакансий, retriever реализует отбор документов по семантической близости, а языковая модель может использоваться для извлечения структурированных ограничений из пользовательского запроса, построения кратких пояснений и повышения интерпретируемости результата. Следовательно, RAG следует рассматривать как общую теоретическую рамку, в которой объединяются модели представления текста, ранжирования и генеративного анализа запроса [2], [9].

1.5 Выводы по главе 1

1. Задача поиска вакансий формализуется как задача информационного поиска, в которой требуется не только найти документы, но и упорядочить их по степени релевантности пользовательскому запросу [1], [2].
2. Эволюция моделей представления текста показывает закономерный переход от лексических методов к контекстным и далее к эмбедингам предложений, что делает использование Sentence-BERT обоснованным для семантического поиска вакансий [10].
3. Для bi-encoder представлений наиболее естественной метрикой служит косинусная близость, а ее программная реализация в pgvector позволяет выполнять ранжирование непосредственно в PostgreSQL [13], [14].
4. Архитектура Retrieval-Augmented Generation задает модель взаимодействия retrieval и generation, в которой внешняя память снижает зависимость от внутренних параметров LLM и повышает обоснованность ответа [9].
5. Рассмотренные теоретические положения образуют основу для дальнейшего проектирования каскада моделей семантического поиска вакансий, включающего векторное представление, поиск по близости, интерпретацию запроса и последующее уточнение выдачи.
6. В рамках данной курсовой работы акцент сделан на реализации первой части RAG-цепочки – высокоточном семантическом поиске, усиленном LLM-интерпретацией запроса. Полноценная генерация текстовых рекомендаций рассматривается как направление дальнейшего развития системы

2 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ПОСТАНОВКА ЗАДАЧИ

2.1 Особенности рынка труда Республики Беларусь и задачи поиска работы соискателями

Поиск работы в Республике Беларусь характеризуется сочетанием двух факторов: высокой вариативностью формулировок вакансий и ограниченным набором характеристик от самих соискателей. Пользователь, как правило, описывает не конкретную должность, а намерение: найти первую работу в информационных технологиях, стажировку в маркетинге, подработку в пункте выдачи заказов или вакансию без опыта. При этом работодатель нередко использует иные формулировки, включая смешение русскоязычной и англоязычной терминологии: junior developer, support specialist, backend engineer, sales manager. Для корпуса вакансий, собираемого с портала rabota.by, это означает необходимость обработки не только точных словесных совпадений, но и смысловой близости запросов и описаний вакансий [6], [7].

Дополнительная сложность предметной области связана со структурой самих карточек вакансий. В проекте каждая вакансия описывается набором полей salary, experience, employment, hiring_format, schedule, hours, work_format, skills, а также полным текстовым описанием. Часть этих полей заполняется строго, однако значительная доля значений представлена в полуформализованном виде: «Не указано», «Не требуется», «Два раза в месяц», «На месте работодателя или разъездной». Следовательно, система должна уметь сочетать работу со структурированными признаками и анализ свободного текста, поскольку именно их совместное использование позволяет восстановить реальное содержание вакансии.

Для рынка труда Республики Беларусь существенна и неоднородность начальных позиций. В корпусе проекта соседствуют вакансии из информационных технологий, логистики, торговли, сферы услуг и административной деятельности. При этом близкие по содержанию роли могут иметь разные названия, а одинаково названные позиции могут относиться к различным предметным контекстам. Например, роли продавца, консультанта, сотрудника пункта выдачи заказов, мерчендайзера или администратора часто пересекаются по отдельным навыкам, но различаются по целевой функции и рабочей среде. Для соискателя без значительного опыта такая неоднозначность усложняет поиск и повышает требования к качеству ранжирования.

В текущей версии проекта используется база из 1016 вакансий. Это позволяет рассматривать задачу не как единичный подбор, а как полноценный информационный поиск по локальному рынку труда. С учетом мультязычных формулировок, вариативности ролей и неполноты полей традиционный поиск по ключевым словам оказывается недостаточным. Отсюда следует постановочный вывод: для рассматриваемой предметной области требуется мультязычная модель семантического сопоставления, дополненная механизмом мягкой обработки пользовательских ограничений.

2.2 Обзор существующих решений и их ограничений

Наиболее релевантным внешним аналогом для разрабатываемой системы выступает портал rabota.by [19], поскольку именно он служит источником фактического набора вакансий и отражает актуальные практики найма в Республике Беларусь. Сильной стороной данного решения является наличие развитой системы фильтров по опыту, занятости, графику и формату работы, а также структурированных карточек вакансий. Однако поисковый сценарий портала в первую очередь ориентирован на явный ввод ключевых слов и ручной подбор фильтров пользователем. Если соискатель формулирует запрос на естественном языке, например описывает желаемую роль, ограничения и условия в одной фразе, семантическая интерпретация запроса остается ограниченной.

Международные платформы LinkedIn, HeadHunter и Indeed предоставляют сопоставимый набор инструментов: поиск по названию должности, фильтрацию по формату занятости, опыту, географии и уровню оплаты. В ряде случаев они дополняются рекомендационными механизмами и профилями пользователя. Тем не менее базовый сценарий взаимодействия по-прежнему опирается на то, что пользователь уже знает, какую роль и какими словами следует искать. Для соискателя это предположение выполняется не всегда, поскольку на старте карьеры описание собственных компетенций чаще задается через навыки, учебные проекты и желаемую область деятельности, а не через точное название вакансии.

Таблица 2.1 – Характерные результаты контрольного тестирования

Критерий	rabota.by	LinkedIn	HeadHunter	Предлагаемый подход
Основной принцип поиска	ключевые слова и фильтры	профиль, вакансии, рекомендации	ключевые слова, фильтры, рекомендации	семантическое сопоставление запроса и вакансий
Работа со свободным запросом	ограниченная	частично поддерживается	ограниченная	поддерживается через LLM-извлечение фильтров
Учет синонимии и неполных формулировок	слабый	частичный	частичный	через эмбединги Sentence-BERT
Поведение при слишком строгих фильтрах	возможна пустая выдача	зависит от рекомендаций	возможна пустая выдача	релаксация фильтров по весам
Объяснимость результата	низкая	частичная	частичная	через сохранение фильтров, role_keywords и шагов релаксации
Адаптация к корпусу Республики Беларусь	высокая	низкая/средняя	средняя	высокая, корпус rabota.by

Примечание – Источник: собственная разработка.

Общий недостаток перечисленных решений состоит в преобладании парадигмы «жесткий фильтр плюс лексический поиск». Во первых, такая схема плохо обрабатывает синонимию, сокращения и смешение русского и

английского профессионального словаря. Во вторых, при последовательном наложении нескольких ограничений система нередко приходит к пустой выборке, хотя на рынке существуют содержательно близкие вакансии, которые отличаются только одним или двумя второстепенными признаками. Для пользователя это выражается либо в отсутствии результатов, либо в необходимости многократно переформулировать запрос.

Следовательно, целесообразно не заменять существующие источники вакансий, а надстраивать над ними интеллектуальный слой интерпретации. В рамках курсовой работы таким слоем выступает каскад моделей, который сохраняет достоинства структурированного поиска по полям, но добавляет семантическое сопоставление текста, автоматическое извлечение фильтров из естественного языка и мягкую релаксацию ограничений в ситуациях нулевой выдачи. Именно эта комбинация отличает разрабатываемое решение от традиционных HR платформ.

2.3 Формулировка задачи и требования к системе

Сформулируем задачу в общем виде. Имеется корпус вакансий V , где каждая запись содержит структурированные атрибуты и текстовое описание, а также пользовательский запрос q , сформулированный на естественном языке. Требуется построить отображение $F(q, V)$, результатом которого является ранжированный список наиболее релевантных вакансий, отражающий как смысловую близость текста, так и соответствие формальным ограничениям по опыту, занятости, графику и формату работы. В прикладном варианте проекта пользователю возвращаются не более пяти вакансий, упорядоченных по степени релевантности.

В рамках постановки задачи выделены три ключевых пользовательских сценария. Сценарий «умный профиль» предполагает, что соискатель описывает свои навыки, знания и проекты свободным текстом. Сценарий «интеллектуальный анализ» требует объяснить, почему найденная вакансия подходит пользователю и какие условия в ней являются значимыми. Сценарий «пять лучших вакансий и краткое резюме» ориентирован на быстрый обзор наиболее подходящих предложений. В текущем прототипе полностью реализован поисковый контур с чат-интерфейсом и серверным маршрутом `GET /api/v1/search`, а сценарии объяснения релевантности и формирования кратких резюме рассматриваются как направления дальнейшего развития системы.

К функциональным требованиям разрабатываемой системы относятся: автоматизированный сбор вакансий с `rabota.by`; нормализация и хранение карточек вакансий в базе данных; построение векторных представлений размерности 384 для последующего семантического поиска; извлечение структурированных фильтров из пользовательского текста при помощи языковой модели; выполнение векторного поиска с учетом фильтров; формирование итоговой выдачи с полными карточками вакансий и ссылками на исходные объявления. Дополнительно система должна поддерживать оценку качества поиска на контрольном наборе запросов, что необходимо для экспериментальной проверки результатов.

К нефункциональным требованиям относятся мультиязычность, устойчивость к неполным и неоднозначным запросам, приемлемая скорость ответа, воспроизводимость результатов и корректная обработка ошибок внешнего LLM сервиса. Для этого в проекте предусмотрены строгая валидация запросов, ограничение числа возвращаемых результатов, повторные попытки обращения к внешнему API и контроль допустимых значений фильтров на основе содержимого базы. Отдельным требованием является предотвращение нулевой выдачи: если исходные ограничения оказываются слишком узкими, система должна ослаблять наименее важные фильтры, сохраняя приоритет ключевых условий пользователя.

По итогам выполнения курсовой работы должен быть получен работоспособный сквозной прототип интеллектуальной системы поиска вакансий с чат-интерфейсом, локальной базой знаний объемом 1016 вакансий и векторным представлением размерности 384. Для системы задаются ориентиры практической эффективности: выдача не более пяти наиболее релевантных вакансий и достижение метрики `hit@5` на эталонном наборе из 16 тестовых запросов на уровне не ниже 0,75. Кроме того, ожидается экспериментальное подтверждение положительного вклада подмоделей каскада.

2.4 Обоснование стека технологий и архитектуры

Выбор стека технологий определяется не только удобством разработки, но и соответствием математической постановке задачи. В качестве основного языка выбран Python версии 3.10 и выше, поскольку он предоставляет зрелую экосистему для обработки естественного языка, асинхронных веб-сервисов, работы с данными и интеграции с предобученными моделями. Разделение

проекта на backend и frontend модули упрощает изоляцию вычислительного контура поиска и пользовательского интерфейса.

Серверная часть реализуется на FastAPI. Выбор данного фреймворка оправдан его поддержкой асинхронной обработки запросов, интеграцией с Pydantic и возможностью формировать типизированный REST интерфейс с единым форматом ошибок [11]. Для рассматриваемой задачи это существенно, поскольку пользовательский запрос должен проходить валидацию, а ответ обязан возвращать строго определенный набор полей вакансии. Использование маршрута GET /api/v1/search соответствует логике легковесного поискового сервиса, к которому может обращаться внешний интерфейс.

В качестве клиентского слоя выбран Streamlit, позволяющий быстро построить реактивный чат-интерфейс без разработки отдельного веб-клиента на JavaScript [12]. Такой выбор оправдан учебным характером проекта: основное внимание сосредоточено не на визуальном дизайне, а на моделях поиска, интерпретации запроса и представлении результатов. При этом Streamlit обеспечивает удобное отображение карточек вакансий, сокращенного описания и пользовательского диалога, что достаточно для демонстрации работы интеллектуальной системы.

Для хранения данных используется PostgreSQL с расширением pgvector [13], [14], [15]. Данное решение объединяет реляционную целостность, удобство SQL фильтрации и возможность нативно хранить поле vector(384), непосредственно участвующее в вычислении косинусного расстояния. Структура таблицы vacancies соответствует предметной области: каждая запись содержит название вакансии, компанию, сведения об оплате, опыте, графике, навыках, исходную ссылку и полное описание. Наличие уникального ограничения по url позволяет исключить дублирование объявлений при повторных запусках процесса сбора [16].

Нейросетевое ядро проекта построено на библиотеке Sentence-Transformers и модели paraphrase-multilingual-MiniLM-L12-v2 [18]. Ее выбор обусловлен мультязычным характером корпуса и фиксированной размерностью эмбединга 384, согласованной со схемой базы данных. В проекте применяется усиление значимых полей вакансии при построении текста для векторизации: название должности повторяется четыре раза, структурированные поля три раза, навыки один раз. Такое решение повышает вклад наиболее информативных признаков и компенсирует размывание смысла длинными описаниями при усреднении представления bi-encoder модели.

LLM-слой реализуется через OpenRouter [17]; в текущей программной реализации для извлечения фильтров используется модель inclusionai/

ling-2.6-flash. В отличие от первоначально намеченного абстрактного генеративного контура, текущая реализация использует языковую модель не для генерации длинных ответов, а для извлечения взвешенных структурированных фильтров из естественного языка пользователя. Далее эти фильтры проходят валидацию, при необходимости уточняются и передаются в поисковый контур. Тем самым достигается разделение retrieval, выполняемого локально в PostgreSQL и Sentence-BERT, и generation уровня, работающего как внешний сервис. Такое архитектурное решение закладывает фундамент для реализации полноценной RAG-системы, реализуя на текущем этапе наиболее значимый компонент – интеллектуальный поиск и фильтрацию на основе языковой модели.

2.5 Выводы по главе 2

1. Выявлены специфические особенности рынка труда Республики Беларусь, требующие мультязычной обработки запросов, учета синонимии и мягкой интерпретации пользовательских ограничений.
2. Показано, что существующие HR платформы эффективны как источники вакансий, однако не решают в полном объеме задачу семантического поиска для соискателей из-за преобладания лексического поиска и жестких фильтров.
3. Сформулированы функциональные и нефункциональные требования к системе, а также зафиксированы ожидаемые результаты по объему корпуса, размерности эмбедингов и качеству поиска.
4. Обоснован выбор технологического стека Python, FastAPI, Streamlit, PostgreSQL, pgvector, Sentence-Transformers и OpenRouter как согласованной архитектурной основы прототипа.

3 ПРОЕКТИРОВАНИЕ И ПРОГРАММНАЯ РЕАЛИЗАЦИЯ КАСКАДА МОДЕЛЕЙ

3.1 Общая архитектура системы

Проектируемая система реализована как трехуровневая программная архитектура, в которой пользовательский интерфейс, серверная логика и хранилище данных разделены по функциональной ответственности. Клиентский уровень построен на Streamlit и отвечает за ввод запроса, отображение списка найденных вакансий и просмотр подробного описания карточки. Серверный уровень реализован на FastAPI и инкапсулирует всю логику каскада моделей: интерпретацию запроса, фильтрацию, релаксацию ограничений, векторный поиск и итоговое ранжирование. На уровне данных используется PostgreSQL с расширением pgvector, где одновременно хранятся структурированные поля вакансии и ее векторное представление.

С архитектурной точки зрения в системе выделяются два независимых потока обработки. Первый поток представляет собой контур наполнения базы знаний: модуль сбора ссылок получает вакансии с портала rabota.by, модуль детального парсинга извлекает нормализованные атрибуты карточек, после чего данные записываются в локальное хранилище и проходят этап векторизации. Второй поток образует пользовательский поисковый контур: текстовый запрос поступает на сервер, затем из него извлекаются структурированные ограничения и ролевые ключевые слова, выполняется смягчение избыточно жестких фильтров, после чего производится поиск ближайших вакансий по косинусной мере и дополнительное доменное реранжирование.

Подобное разделение делает систему устойчивой и управляемой с инженерной точки зрения. Вычислительно затратные операции сбора и построения эмбедингов вынесены из пользовательского сценария и выполняются заранее, тогда как в интерактивном режиме остаются только операции интерпретации запроса и поиска по уже подготовленному индексу. В результате достигается согласование требований к качеству семантического сопоставления и воспроизводимости эксперимента.

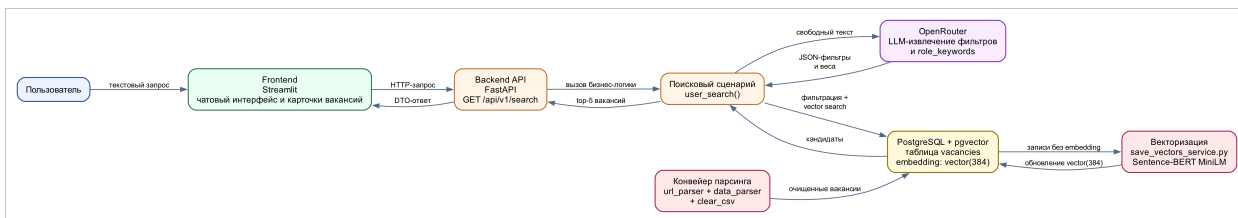


Рисунок 3.1 – Архитектура системы семантического поиска вакансий

Примечание – Источник: собственная разработка.

3.2 Модель хранения знаний: схема базы данных и векторное поле

Центральным элементом хранилища выступает сущность `Vacancy`, описанная в ORM модели проекта. Каждая запись содержит название вакансии, наименование компании, сведения об оплате, формате выплат, требуемом опыте, виде занятости, форме трудоустройства, графике, продолжительности рабочего дня, формате работы, перечне навыков, исходной ссылке и полном описании. Такое представление сочетает структурированные признаки, пригодные для точной фильтрации, и текстовые поля, необходимые для последующего семантического анализа.

Для обеспечения быстрого поиска в проекте выбрана денормализованная схема хранения с одной основной таблицей вакансий. Для рассматриваемого учебного прототипа это решение оказывается оправданным, поскольку позволяет избегать множественных соединений таблиц во время интерактивного поиска и хранить все атрибуты документа в одной строке. Дополнительным средством обеспечения целостности является уникальное ограничение по полю `url`, исключающее повторную загрузку одной и той же вакансии при повторном запуске контура сбора данных.

Ключевым элементом схемы является поле `embedding` размерности 384, создаваемое в миграции базы данных после подключения расширения `vector`. Именно это поле служит носителем плотного представления вакансии в векторном пространстве и используется при вычислении косинусного расстояния на стороне СУБД. Выбор хранения векторов непосредственно в PostgreSQL обеспечивает единое пространство выполнения: структурированные ограничения, подсчет допустимых значений и семантическое ранжирование выполняются в рамках одной базы данных без промежуточной синхронизации между разными хранилищами.

vacancies	
id	SERIAL, PK
title	VARCHAR(512)
company	VARCHAR(512)
salary	VARCHAR(256)
payment_frequency	VARCHAR(128)
experience	VARCHAR(256)
employment	VARCHAR(256)
hiring_format	VARCHAR(256)
schedule	VARCHAR(256)
hours	VARCHAR(256)
work_format	VARCHAR(256)
skills	TEXT
url	VARCHAR(2048), UNIQUE
description	TEXT
embedding	vector(384)

Рисунок 3.2 – Схема сущности Vacancy и векторного поля embedding

Примечание – Источник: собственная разработка.

```

from alembic import op

revision = "0001_create_vacancies"
down_revision = None
branch_labels = None
depends_on = None

def upgrade() -> None:
    op.execute("CREATE EXTENSION IF NOT EXISTS vector")

    op.execute(

```

```

"""
CREATE TABLE vacancies (
    id SERIAL PRIMARY KEY,
    title VARCHAR(512) NOT NULL,
    company VARCHAR(512) NOT NULL,
    salary VARCHAR(256) NOT NULL,
    payment_frequency VARCHAR(128) NOT NULL,
    experience VARCHAR(256) NOT NULL,
    employment VARCHAR(256) NOT NULL,
    hiring_format VARCHAR(256) NOT NULL,
    schedule VARCHAR(256) NOT NULL,
    hours VARCHAR(256) NOT NULL,
    work_format VARCHAR(256) NOT NULL,
    skills TEXT NOT NULL,
    url VARCHAR(2048) NOT NULL,
    description TEXT NOT NULL,
    embedding vector(384),
    CONSTRAINT uq_vacancies_url UNIQUE (url)
)
"""

def downgrade() -> None:
    op.execute("DROP TABLE IF EXISTS vacancies")

```

Листинг 3.1 – Фрагмент миграции создания таблицы vacancies

Примечание – Источник: собственная разработка.

3.3 Модель сбора данных: парсер вакансий rabota.by

Формирование базы знаний реализовано в виде двух последовательных программных модулей. На первом этапе модуль `url_parser.py` обращается к страницам поисковой выдачи `rabota.by`, формирует параметры запроса, извлекает прямые ссылки на карточки вакансий и нормализует их путем удаления лишних параметров адреса. Такой подход позволяет получить компактный и устойчивый список уникальных URL, пригодных для дальнейшего обхода. Параллельно обеспечивается инкрементный режим работы: уже сохраненные ссылки повторно не обрабатываются.

На втором этапе модуль `data_parser.py` последовательно загружает каждую карточку вакансии и извлекает из HTML документа основные поля при помощи `data-qa` атрибутов. В результате для каждой записи формируется единая строка с названием, компанией, оплатой, опытом, типом занятости, навыками, ссылкой и полным текстовым описанием. Такое решение важно для воспроизводимости, поскольку не зависит от визуального представления

страницы и опирается на более стабильные семантические маркеры интерфейса сайта.

С инженерной точки зрения модуль сбора данных содержит ряд механизмов устойчивости к внешним сбоям. В проекте предусмотрены повторные попытки HTTP-запросов с увеличивающейся задержкой, случайные паузы между обращениями, подмена User-Agent и пошаговое сохранение результатов в CSV-файлы после обработки очередной порции данных. Благодаря этому даже при сетевых сбоях или частичных ошибках загрузки накопленные результаты не теряются, а база знаний может наращиваться постепенно.



Рисунок 3.3 – Конвейер сбора и обработки вакансий с портала rabota.by

Примечание – Источник: собственная разработка.

```
# features/parser/file_orchestrator.py
async def orchestrate_parser_pipeline_async(
    pages_to_parse: int = DEFAULT_PAGES_TO_PARSE,
    vacancy_limit: int = DEFAULT_VACANCY_LIMIT,
    links_file: Path = VACANCY_LINKS_FILE,
    details_file: Path = VACANCY_DETAILS_FILE,
    cleaned_file: Path = VACANCIES_FILE,
) -> Any:
    if not has_data(links_file):
        parse_vacancy_links(pages_to_parse=pages_to_parse,
                             output_file=links_file)
    else:
        print(f"Использую существующий файл ссылок:
{links_file}")

    parse_vacancy_details(
        input_file=links_file,
        output_file=details_file,
        limit=vacancy_limit,
    )

    cleaned_data = process_csv(input_file=details_file,
                              output_file=cleaned_file)
    affected = await upsert_cleaned_vacancies(cleaned_data)
    print(f"Upsert в PostgreSQL завершён. Затронуто строк:
{affected}")
    return cleaned_data

# features/parser/url_parser.py
```

```

def parse_vacancy_links(
    pages_to_parse: int = DEFAULT_PAGES_TO_PARSE,
    output_file: Path = VACANCY_LINKS_FILE,
    start_page: int = DEFAULT_START_PAGE,
) -> list[str]:
    user_agent = build_user_agent()
    all_links = read_existing_links(output_file)
    seen_links = set(all_links)
    pages = range(start_page, start_page + pages_to_parse)
    for page in track_progress(pages, total=pages_to_parse,
description="C6op url"):
        page_links = parse_search_page(page, user_agent)
        if page_links is None:
            continue
        if not page_links:
            break
        new_links = [link for link in page_links if link not in
seen_links]
        if not new_links:
            continue
        all_links.extend(new_links)
        seen_links.update(new_links)
        save_links_to_csv(all_links, output_file)
        time.sleep(random.uniform(REQUEST_DELAY_MIN_SECONDS,
REQUEST_DELAY_MAX_SECONDS))
    return all_links

# features/parser/data_parser.py
def parse_vacancy(url: str, user_agent: str) -> dict[str, str] |
None:
    response_text = fetch_vacancy_page(url, user_agent)
    if response_text is None:
        return None
    soup = BeautifulSoup(response_text, DEFAULT_HTML_PARSER)
    return {
        "title": get_text_safe(soup, "vacancy-title"),
        "company": get_text_safe(soup, "vacancy-company-name"),
        "salary": get_text_safe(soup, "vacancy-salary"),
        "payment_frequency": get_text_safe(soup, "compensation-
frequency-text"),
        "experience": get_text_safe(soup, "vacancy-experience"),
        "employment": get_text_safe(soup, "common-employment-
text"),
        "hiring_format": get_text_safe(soup, "vacancy-hiring-
formats"),
        "schedule": get_text_safe(soup, "work-schedule-by-days-
text"),
        "hours": get_text_safe(soup, "working-hours-text"),
        "work format": get_text_safe(soup, "work-formats-text"),
        "skills": parse_skills(soup),
        "url": url,
        "description": parse_description(soup),
    }

# features/parser/clear_csv.py
def process_csv(
    input_file: Path = VACANCY_DETAILS_FILE,
    output_file: Path = VACANCIES_FILE,

```

```

) -> Any:
    output_file.parent.mkdir(parents=True, exist_ok=True)
    try:
        cleaned_data = process_csv_with_pandas(input_file,
output_file)
    except ModuleNotFoundError:
        cleaned_data = process_csv_with_csv(input_file,
output_file)
    print(f"Очистка завершена! Файл сохранен как:
{output_file}")
    return cleaned_data

```

Листинг 3.2 – Фрагмент модуля сбора ссылок и парсинга вакансий `rabota.by`

Примечание – Источник: собственная разработка.

3.4 Модель векторизации вакансии: взвешенное текстовое представление

Ключевой частью проекта является модель подготовки текста вакансии к последующей векторизации. Наивная конкатенация всех полей в одну строку приводит к тому, что длинные шаблонные описания работодателей начинают доминировать над короткими, но семантически наиболее значимыми признаками, прежде всего названием должности и режимом занятости. Для bi-encoder модели семейства Sentence-BERT это особенно важно, поскольку итоговый вектор формируется как усредненное представление токенов, а значит избыточные фрагменты могут размывать смысловой центр документа.

В модуле `save_vectors_service.py` данная проблема решена через контролируемое повторение наиболее значимых полей. Название должности включается в итоговый текст четыре раза, структурированные поля занятости, графика, часов работы, формата работы, опыта и формы трудоустройства повторяются три раза, сведения о компании добавляются один раз, навыки также включаются один раз. Дополнительно в текст попадают сведения о зарплате и формате выплат, а описание вакансии присоединяется как завершающий фрагмент. Тем самым формируется не произвольная строка, а специальное взвешенное представление документа, ориентированное на задачу поиска.

После формирования текста модуль `encoder.py` выполняет пакетное кодирование при помощи модели `paraphrase-multilingual-MiniLM-L12-v2`. Полученные векторные представления размерности 384 синхронно сохраняются в поле `embedding` соответствующих записей. Полный листинг

ключевого фрагмента векторизации приведен в приложении А. Практическое значение такой схемы состоит в том, что семантическое пространство начинает лучше отражать именно профессиональную роль вакансии, а не только общий фон описания работодателя.

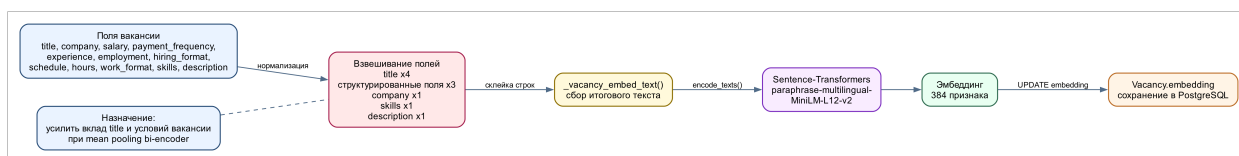


Рисунок 3.4 – Схема формирования взвешенного текстового представления вакансии

Примечание – Источник: собственная разработка.

```

_STRUCTURE_FIELDS: tuple[tuple[str, str], ...] = (
    ("Занятость", "employment"),
    ("График", "schedule"),
    ("Часы", "hours"),
    ("Формат работы", "work_format"),
    ("Опыт", "experience"),
    ("Трудоустройство", "hiring_format"),
)

def _repeat_lines(label: str, value: str, times: int) ->
list[str]:
    if _skip_value(value):
        return []
    line = f"{label}: {_trim(value)}"
    return [line] * times

def _description_for_embed(raw: str) -> str | None:
    if not VACANCY_EMBED_INCLUDE_DESCRIPTION:
        return None
    t = _trim(raw)
    if not t:
        return None
    cap = VACANCY_EMBED_DESCRIPTION_MAX_CHARS
    if cap is not None and len(t) > cap:
        return t[:cap].rstrip() + "..."
    return t

def _vacancy_embed_text(row: Any) -> str:
    chunks: list[str] = []

    title = _trim(row.title)
    if title:

```

```

        job_line = f"Должность: {title}"
        chunks.extend([job_line] *
VACANCY_EMBED_TITLE_REPEATS)

        chunks.extend(_repeat_lines("Компания", row.company,
VACANCY_EMBED_COMPANY_REPEATS))
        for label, attr in _STRUCTURE_FIELDS:
            chunks.extend(
                _repeat_lines(
                    label,
                    getattr(row, attr),
                    VACANCY_EMBED_STRUCTURE_REPEATS,
                )
            )
        chunks.extend(_repeat_lines("Зарплата", row.salary,
1))
        chunks.extend(_repeat_lines("Выплаты",
row.payment_frequency, 1))

        if not _skip_value(row.skills):
            sk = f"Навыки: {_trim(row.skills)}"
            chunks.extend([sk] *
VACANCY_EMBED_SKILLS_REPEATS)

        desc = _description_for_embed(row.description)
        if desc:
            chunks.append(f"Описание: {desc}")

        return "

".join(chunks)

```

Листинг 3.3 – Фрагмент функции формирования взвешенного текстового представления вакансии

Примечание – Источник: собственная разработка.

3.5 Модель извлечения структурированных фильтров через LLM

Следующим звеном каскада является преобразование пользовательского запроса на естественном языке в набор структурированных ограничений. В проекте эта задача решается при помощи внешней языковой модели, вызываемой через OpenRouter. Пользователь может сформулировать запрос свободно, например указать желаемую роль, отсутствие опыта, формат занятости и предпочтительный график одной фразой. Для классического фильтрационного интерфейса такой ввод неудобен, тогда как LLM-слой интерпретирует его как источник семантически связанных ограничений.

Модуль `llm/service.py` сначала строит множество допустимых значений для каждого фильтрующего поля на основе реального содержимого базы данных. Затем в системный промпт подставляется JSON со всеми разрешенными вариантами, правилами нормализации синонимов и требованием вернуть строго валидный JSON-объект. В текущей реализации извлекаются семь групп фильтров: формат выплат, опыт, вид занятости, форма трудоустройства, график, часы работы и формат работы. Дополнительно языковая модель возвращает массив `role_keywords`, содержащий короткие названия предполагаемых ролей или профессиональных доменов.

Особенностью данной модели является использование не только значений, но и весов важности. Для каждого выбранного ограничения LLM возвращает число от нуля до единицы, отражающее степень критичности данного признака для пользователя. После получения ответа модуль выполняет строгую валидацию структуры JSON, типов полей, диапазона весов и принадлежности значений к допустимому множеству. При нарушении хотя бы одного требования запускается повторный цикл запроса с передачей ошибок валидации обратно в контекст модели. Такой механизм делает слой интерпретации управляемым и пригодным для дальнейшего алгоритмического использования. Полный листинг промпта и логики валидации вынесен в приложение А.

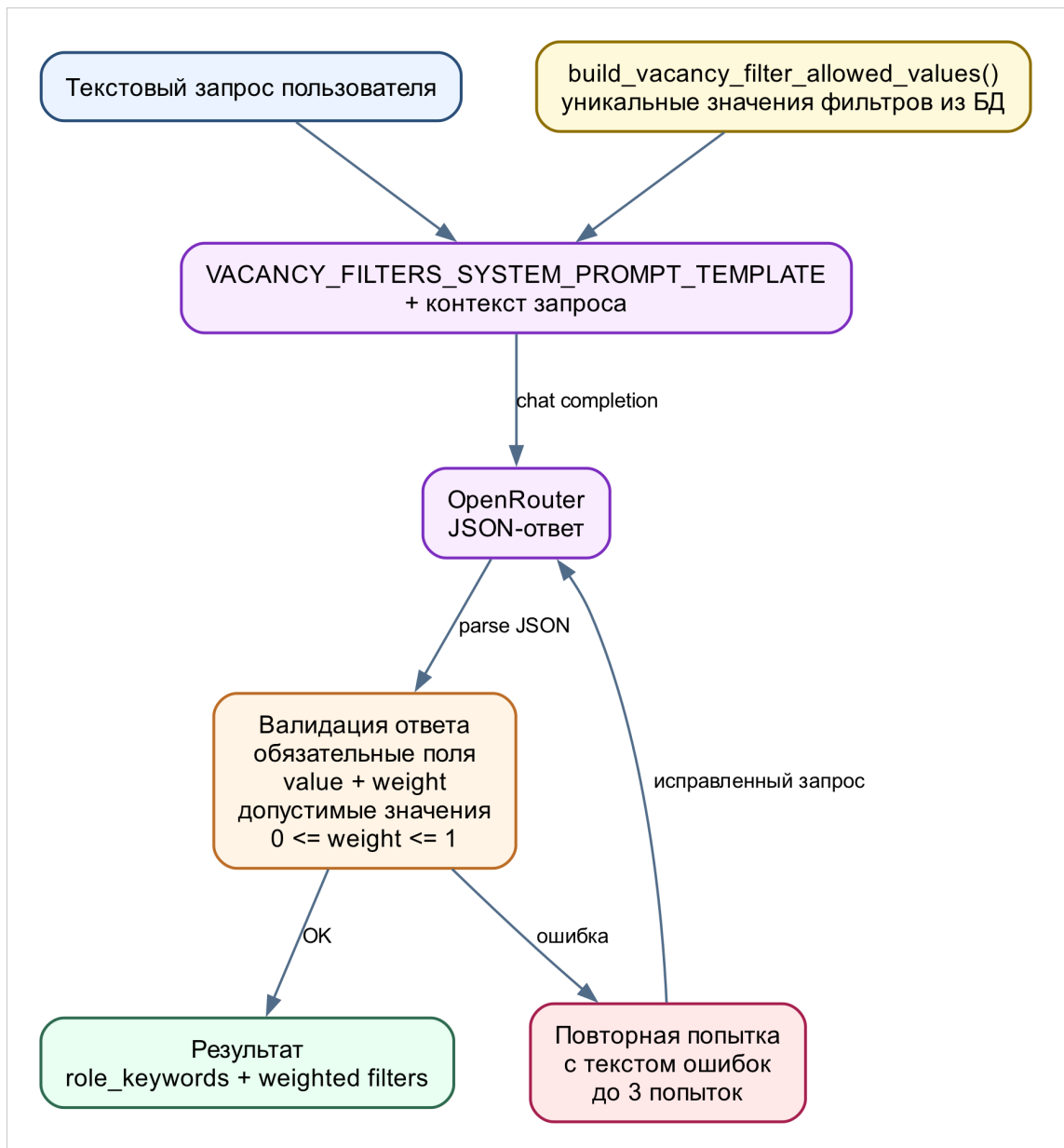


Рисунок 3.5 – Схема извлечения и валидации структурированных фильтров через LLM

Примечание – Источник: собственная разработка.

```

def _validate_llm_payload(
    payload: dict[str, Any], allowed_values: dict[str,
list[str]]
) -> list[str]:
    errors: list[str] = []
    required = set(VACANCY_FILTER_KEYS)
    optional = set(VACANCY_OPTIONAL_KEYS)
    payload_keys = set(payload.keys())
    if not required.issubset(payload_keys):
        errors.append("JSON must contain all required filter

```

```

keys.")

    role_keywords = payload.get("role_keywords", [])
    if not isinstance(role_keywords, list):
        errors.append("Field 'role_keywords' must be an array.")
        elif not all(isinstance(item, str) for item in
role_keywords):
            errors.append("Field 'role_keywords' must contain only
strings.")

    for key in VACANCY_FILTER_KEYS:
        value = payload.get(key)
        if not isinstance(value, list):
            errors.append(f"Field '{key}' must be an array.")
            continue
        allowed_set = set(allowed_values[key])
        for item in value:
            if not isinstance(item, dict):
                errors.append(f"Field '{key}' items must be
objects.")
                continue
            if set(item.keys()) != {"value", "weight"}:
                errors.append(
                    f"Field '{key}' items must contain only
'value' and 'weight'."
                )
                continue
            raw_filter_value = item.get("value")
            raw_weight = item.get("weight")
            if not isinstance(raw_filter_value, str):
                errors.append(f"Field '{key}' item 'value' must
be a string.")
                continue
            if not isinstance(raw_weight, int | float):
                errors.append(f"Field '{key}' item 'weight' must
be numeric.")
                continue
            if not 0 <= float(raw_weight) <= 1:
                errors.append(
                    f"Field '{key}' item 'weight' must be in
range [0, 1]."
                )
                continue
            if raw_filter_value not in allowed_set:
                errors.append(
                    f"Field '{key}' has invalid values:
{raw_filter_value}."
                )
        return errors

async def get_vacancy_filters_from_text_async(user_message: str)
-> dict[str, Any]:
    allowed_values = await build_vacancy_filter_allowed_values()
    prompt = _build_system_prompt(allowed_values)
    base_context = VACANCY_FILTERS_CONTEXT_TEMPLATE.format(
        user_message=user_message.strip()
    )
    context = base_context
    errors: list[str] = []

```

```

for _ in range(LLM_VALIDATION_RETRY_ATTEMPTS):
    try:
        answer = get_llm_answer_service(prompt, context)
    except ValueError as error:
        errors = [str(error)]
    else:
        errors = _validate_llm_payload(answer.payload,
allowed_values)
        if not errors:
            return answer.payload
        context = _build_retry_context(base_context, errors)
        details = "; ".join(errors) if errors else "Unknown
validation error."
        raise RuntimeError(f"LLM response validation failed:
{details}")

```

Листинг 3.4 – Фрагмент извлечения и валидации структурированных фильтров через LLM

Примечание – Источник: собственная разработка.

3.6 Модель релаксации фильтров

Прямое применение всех ограничений, извлеченных языковой моделью, нередко приводит к чрезмерному сужению выборки. Для рынка вакансий это особенно характерно, поскольку пользователь может одновременно указать желаемую роль, отсутствие опыта, конкретный формат занятости и график, тогда как в реальном корпусе часть этих признаков распределена неравномерно. Если система в такой ситуации просто вернет пустой результат, семантические преимущества всей архитектуры будут фактически утрачены.

Для предотвращения нулевой выдачи в модуле `search/service.py` реализован жадный алгоритм релаксации фильтров. Сначала для полного набора ограничений вычисляется число кандидатов в базе знаний. Если этого числа недостаточно, система строит порядок удаления фильтров по принципу минимальной важности. Вес каждого поля берется как максимальный вес значений внутри данного поля, после чего для критически значимых признаков вводится дополнительный защитный штраф. В текущей конфигурации такими признаками выступают опыт и вид занятости, поэтому они удаляются в последнюю очередь.

На каждом шаге алгоритм исключает наименее важное поле, повторно подсчитывает число подходящих записей и фиксирует шаг релаксации в специальной структуре `FilterRelaxStep`. Тем самым обеспечивается

прозрачность принятого решения: можно проследить, какое ограничение было снято, каков был его вес и насколько изменилось число кандидатов в базе. Практически это означает переход от жесткой логики фильтрации к мягкой деградации условий, при которой пользователь все равно получает содержательно близкие вакансии даже при избыточно узком исходном запросе.

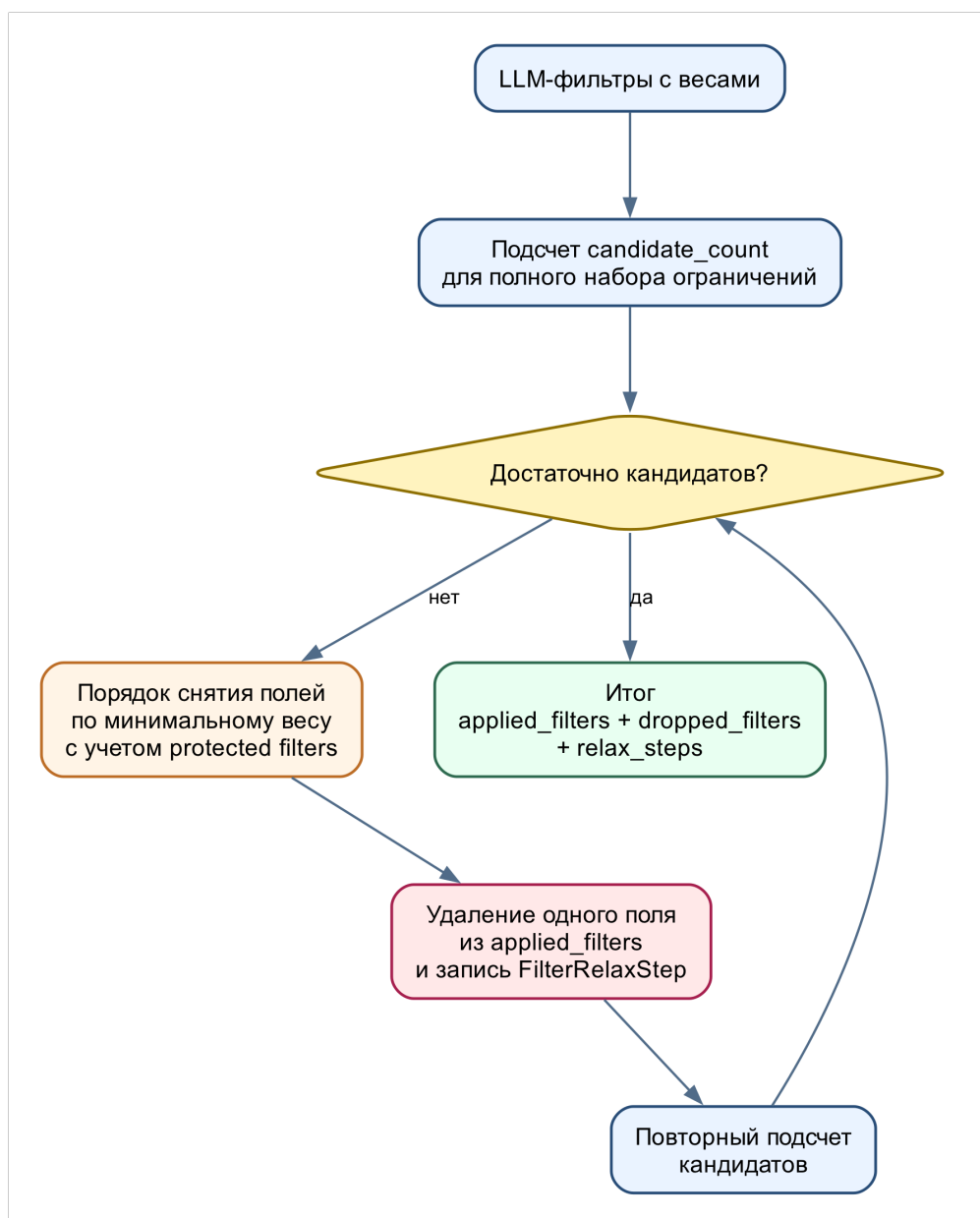


Рисунок 3.6 – Схема алгоритма релаксации фильтров

Примечание – Источник: собственная разработка.

```

    def _build_field_drop_order(
        weighted_filters: dict[str, list[tuple[str, float]]],
        protected_keys: set[str],
    ) -> list[tuple[str, float]]:
        fields: list[tuple[str, float]] = []
        for key in VACANCY_FILTER_KEYS:
            values = weighted_filters[key]
            if values:
                field_weight = max(weight for _, weight in values)
                fields.append((key, field_weight))

        def _effective_weight(item: tuple[str, float]) -> float:
            key, weight = item
            if key in protected_keys:
                return min(1.0, weight +
PROTECTED_RELAX_WEIGHT_PENALTY)
            return weight

        return sorted(
            fields,
            key=lambda item: (_effective_weight(item), item[0]),
        )

async def _relax_filters(
    weighted_filters: dict[str, list[tuple[str, float]]],
    *,
    value_counts: dict[str, dict[str, int]],
    role_keywords: list[str],
    min_candidates: int,
) -> tuple[dict[str, list[str]], dict[str, list[str]],
list[FilterRelaxStep], int]:
    active_filters =
_clone_non_empty(_plain_filters_from_weighted(weighted_filters))
    weighted_by_key = {key: list(values) for key, values in
weighted_filters.items()}
    dropped_filters = empty_filters()
    relax_steps: list[FilterRelaxStep] = []
    candidate_count = await _count_filtered_candidates(
        active_filters, role_keywords=role_keywords
    )
    if candidate_count >= min_candidates:
        return _with_all_keys(active_filters), dropped_filters,
relax_steps, candidate_count

    protected_keys = set(PROTECTED_RELAX_FILTER_KEYS)
    drop_order = _build_field_drop_order(weighted_by_key,
protected_keys)
    for step_index, (key, weight) in enumerate(drop_order,
start=1):
        removed_values = [value for value, _ in
weighted_by_key.get(key, [])]
        if not removed_values:
            continue
        candidates_before = candidate_count
        dropped_filters[key].extend(removed_values)
        active_filters.pop(key, None)
        candidate_count = await _count_filtered_candidates(
            active_filters, role_keywords=role_keywords
        )

```

```

        removed_total_count = sum(
            value_counts.get(key, {}).get(value, 0) for value in
removed_values
        )
        relax_steps.append(
            FilterRelaxStep(
                step=step_index,
                filter_key=key,
                filter_value=", ".join(removed_values),
                removed_weight=weight,
                value_count_in_db=removed_total_count,
                candidates_before=candidates_before,
                candidates_after=candidate_count,
            )
        )
        if candidate_count >= min_candidates:
            break
        return _with_all_keys(active_filters), dropped_filters,
relax_steps, candidate_count

```

Листинг 3.5 – Фрагмент алгоритма релаксации фильтров

Примечание – Источник: собственная разработка.

3.7 Модель гибридного ранжирования с доменным бустом

После применения структурированных ограничений и их возможной релаксации система переходит к основному этапу семантического поиска. Модуль `similarity_search_service.py` строит вектор пользовательского запроса той же размерности, что и вектор вакансии, и затем выполняет ранжирование по косинусному расстоянию непосредственно в PostgreSQL. Дополнительно в запрос может включаться жесткая проверка `role_keywords` по полям `title` и `skills`, что позволяет удерживать поиск внутри предполагаемой профессиональной области.

Однако одной только косинусной близости оказывается недостаточно в случаях, когда несколько вакансий имеют схожий общий смысл, но относятся к различным доменам. Для решения этой проблемы в модуле `search/service.py` реализована модель доменного буста. Сначала из пользовательского запроса и массива `role_keywords` формируется набор доменных ключевых слов. Далее для каждой вакансии подсчитывается число совпадений этих слов в названии и перечне навыков. Полученное число используется как корректирующий фактор для косинусного расстояния: чем больше доменных совпадений, тем выше итоговая позиция вакансии в выдаче.

В текущей реализации доменный буст задается как небольшой сдвиг расстояния для групп ключевых слов, относящихся, например, к сфере

horeca, информационным технологиям и логистике. Такой механизм не требует повторного пропуска пары запрос-документ через отдельный cross-encoder, но при этом улучшает порядок верхней части выдачи в тех случаях, когда роль пользователя выражена явно. Следовательно, финальный этап каскада можно интерпретировать как легковесную доменную модель реранжирования, повышающую точность без существенного роста вычислительных затрат. Полный листинг алгоритма также приведен в приложении А.

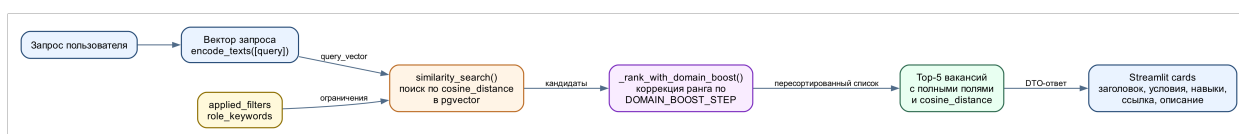


Рисунок 3.7 – Схема гибридного ранжирования с доменным бустом

Примечание – Источник: собственная разработка.

```
def _query_domain_keywords(query: str, role_keywords:
list[str]) -> set[str]:
    query_lower = query.lower()
    matched: set[str] = set(role_keywords)
    for keywords in _DOMAIN_KEYWORDS.values():
        if any(keyword in query_lower for keyword in keywords):
            matched.update(keywords)
    return matched

def _rank_with_domain_boost(
    user_query: str,
    role_keywords: list[str],
    vacancies: list[SimilaritySearchResult],
) -> list[SimilaritySearchResult]:
    keywords = _query_domain_keywords(user_query, role_keywords)
    if not keywords:
        return vacancies
    scored: list[tuple[float, float, SimilaritySearchResult]] =
[]
    for vacancy in vacancies:
        haystack = f"{vacancy.title} {vacancy.skills}".lower()
        matches = sum(1 for keyword in keywords if keyword in
haystack)
        boosted_distance = (
            vacancy.cosine_distance - DOMAIN_BOOST_STEP *
min(matches, 3)
        )
        scored.append((boosted_distance,
vacancy.cosine_distance, vacancy))
```

```
scored.sort(key=lambda item: (item[0], item[1]))  
return [item[2] for item in scored]
```

Листинг 3.6 – Фрагмент алгоритма доменного реранжирования результатов

Примечание – Источник: собственная разработка.

3.8 Реализация клиентского и серверного слоев

Серверная интеграция каскада моделей выполнена через REST-интерфейс FastAPI. Пользовательский запрос поступает на маршрут GET /api/v1/search, где проходит предварительную проверку длины и корректности. После этого запускается единый сценарий `user_search`, объединяющий вызов LLM, построение прикладных фильтров, релаксацию ограничений, поиск по эмбедингам и доменное реранжирование. Возвращаемый ответ представляет собой типизированный список вакансий с полным набором атрибутов, что упрощает дальнейшее отображение результата на клиенте.

Важным элементом серверной реализации является единый формат ошибок и строгая типизация схем ответа. Это уменьшает связанность между интерфейсом и внутренней логикой, а также делает поведение системы предсказуемым при ошибках внешнего LLM-сервиса или некорректном пользовательском вводе. Иначе говоря, каскад моделей встроен в программную оболочку не как экспериментальный скрипт, а как полноценный сервисный сценарий, пригодный для многократного воспроизведения.

Клиентский уровень реализован в виде Streamlit-приложения с чатовой формой взаимодействия. После ввода запроса пользователь получает до пяти карточек вакансий, где отображаются название, компания, ключевые условия, навыки, ссылка на исходное объявление и разворачиваемое описание. Такой интерфейс соответствует учебной цели проекта: он не перегружает пользователя сложными настройками, но в наглядной форме демонстрирует результат совместной работы всех моделей каскада, начиная от интерпретации запроса и заканчивая выдачей итогового списка вакансий.

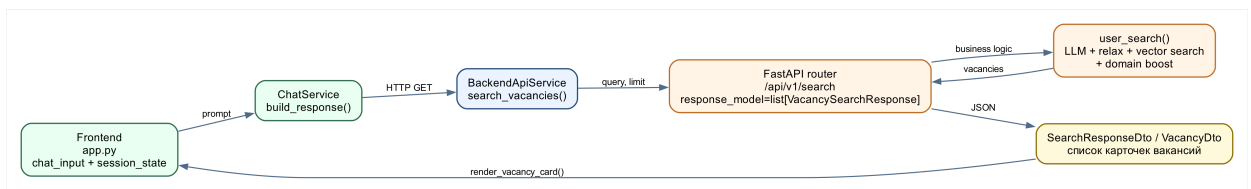


Рисунок 3.8 – Схема взаимодействия клиентского и серверного слоев системы

Примечание – Источник: собственная разработка.

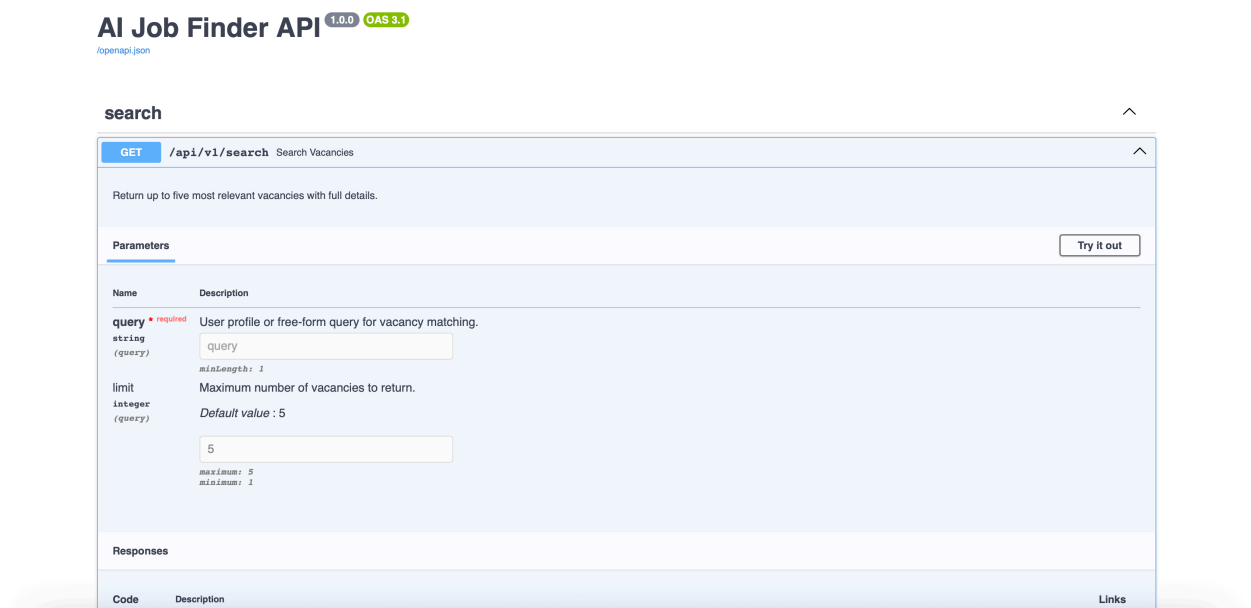


Рисунок 3.9 – Swagger-описание endpoint маршрута GET /api/v1/search

Примечание – Источник: собственная разработка.

3.9 Выводы по главе 3

1. Спроектирована трехуровневая архитектура интеллектуальной системы, разделяющая клиентский, серверный и хранилищный уровни, а также контуры наполнения базы знаний и пользовательского поиска.
2. Реализована модель хранения вакансий в PostgreSQL с полем embedding размерности 384 и уникальным ограничением по адресу

- вакансии, что обеспечивает целостность и готовность данных к семантическому поиску.
3. Построены программные модели сбора данных и взвешенной векторизации, позволяющие преобразовывать вакансии `rabota.by` в устойчивое семантическое представление, ориентированное на профессиональную роль и условия труда.
 4. Реализована LLM-модель извлечения структурированных фильтров с весами важности, строгой валидацией и повторными попытками генерации корректного JSON-ответа.
 5. Разработана модель релаксации фильтров, предотвращающая нулевую выдачу за счет контролируемого снятия наименее важных ограничений с сохранением приоритета критичных признаков.
 6. Завершающий этап каскада реализован в виде векторного поиска и доменного реранжирования, что позволяет формировать интерпретируемую и практически полезную итоговую выдачу для пользователя.

4 ЭКСПЕРИМЕНТАЛЬНАЯ ОЦЕНКА И АНАЛИЗ РАБОТЫ СИСТЕМЫ

4.1 Постановка эксперимента и методика оценки

Цель экспериментальной части состояла в количественной проверке того, насколько разработанный прототип способен возвращать практически релевантные вакансии по свободно сформулированному пользовательскому запросу. В этой части оценивалось качество итогового ранжирования в условиях реальных формулировок соискателя, содержащих одновременно профессиональную роль, желаемый формат работы, требования к графику и ограничения по опыту.

В качестве бенчмарка (эталонного тестирования) использовался встроенный модуль `evaluation.py`, входящий в серверную часть проекта. В нем задано 16 контрольных запросов, охватывающих несколько типовых сценариев: поиск работы в пункте выдачи заказов, вакансии бариста, складские и логистические роли, `support` и `help desk` позиции, вакансию программиста 1С, официанта, кондитера, ассистента интернет-маркетолога, торгового представителя и другие прикладные случаи. Такой набор позволяет проверить как узкоспециализированные, так и неоднозначные запросы, для которых простое совпадение ключевых слов обычно оказывается недостаточным.

Для каждого тестового запроса вручную задавался набор ожидаемых ключевых слов, отражающих релевантную профессиональную роль. Оценивание производилось двумя метриками. Метрика `hit@1` принимала значение 1, если первая вакансия в выдаче содержала ожидаемый профессиональный маркер в названии или навыках. Метрика `hit@5` фиксировала наличие хотя бы одной релевантной вакансии в пределах первых пяти результатов. Дополнительно анализировалось косинусное расстояние `top-1` результата: чем оно меньше, тем ближе найденная вакансия к смысловому вектору запроса.

Эксперимент выполнялся на том же программном контуре, который используется в прикладном режиме: запрос сначала проходит через LLM-слой извлечения фильтров и `role_keywords`, затем при необходимости активируется модель релаксации, после чего выполняется векторный поиск по `pgvector` и завершающее доменное реранжирование. Таким образом, измерялась не отдельная локальная функция, а поведение полного пользовательского пайплайна (цепочки этапов или процессов).

4.2 Функциональное тестирование системы в пользовательских сценариях

На рисунке 4.1 показан сценарий поиска работы в пункте выдачи заказов с упоминанием Ozon и Wildberries. Система возвращает пять вакансий и уже на первой позиции размещает вакансию «Продавец-кассир в пункт выдачи заказов, город Гродно», что можно считать корректным попаданием в целевую роль. Данный пример подтверждает, что модель успешно распознает бытовую формулировку ПВЗ и связывает ее с релевантной категорией вакансий.

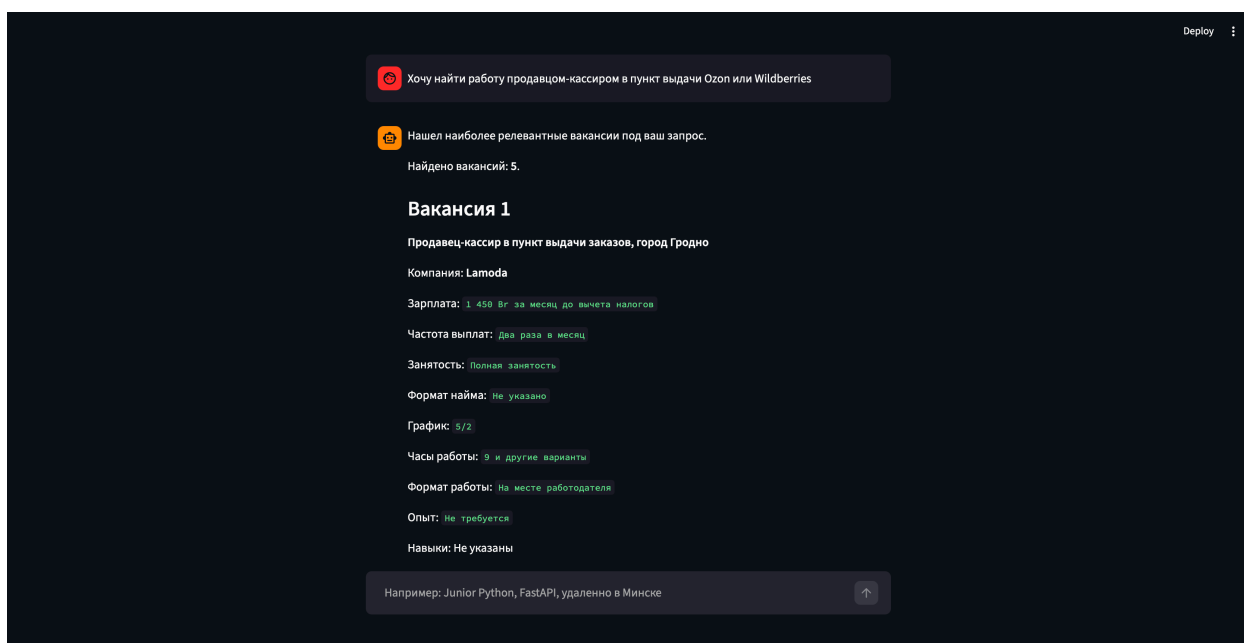


Рисунок 4.1 – Интерфейс поиска по запросу о работе в пункте выдачи заказов

Примечание – Источник: собственная разработка.

На рисунке 4.2 представлен запрос студента на стажировку в сфере разработки программного обеспечения без опыта работы. В ответ система предлагает вакансию «Бизнес-аналитик/Специалист по внедрению ПО», для которой указан опыт от одного до трех лет, то есть совпадение по IT-домену достигается, но требования пользователя удовлетворяются лишь частично. Этот сценарий показывает ограничение текущего ранжирования: семантическая близость к области разработки оказывается выше, чем учет признаков стажировки и отсутствия опыта.

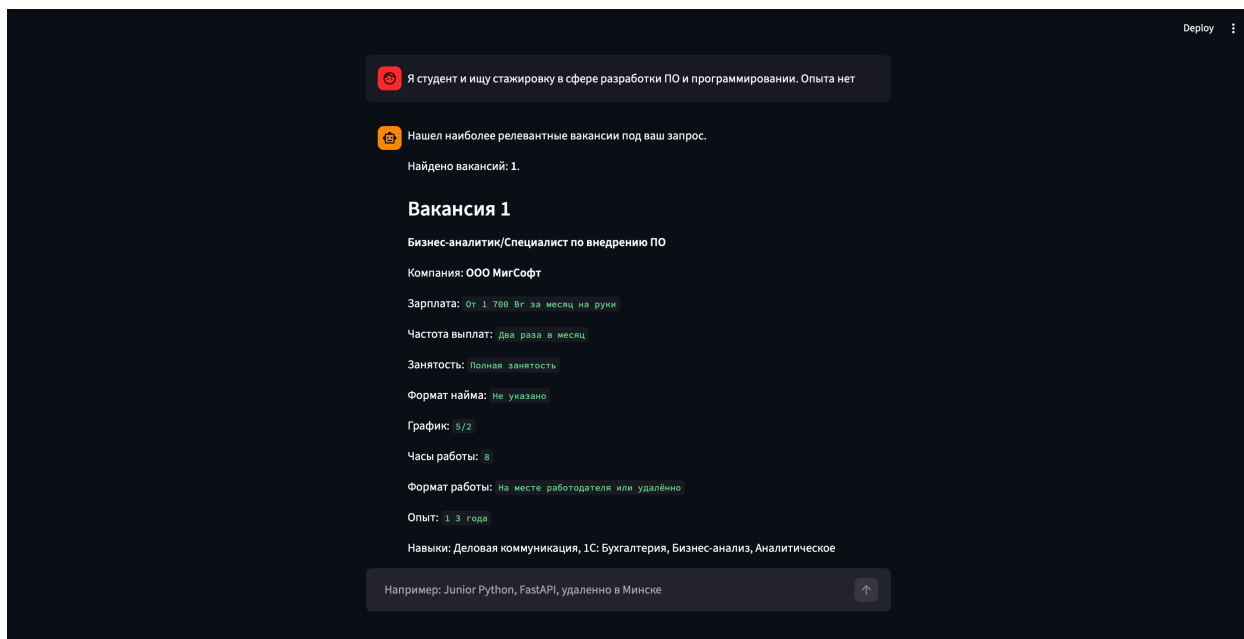


Рисунок 4.2 – Интерфейс поиска по запросу студента на стажировку в IT

Примечание – Источник: собственная разработка.

На рисунке 4.3 отражен запрос на удаленную работу интернет-маркетолога. В top-1 система возвращает вакансию «Ассистент интернет-маркетолога» с удаленным форматом работы, что демонстрирует корректную интерпретацию как профессиональной роли, так и режима занятости. Показательно, что найденная позиция относится к начальному уровню и не требует опыта, поэтому данный пример можно рассматривать как одно из наиболее точных срабатываний интерфейса.

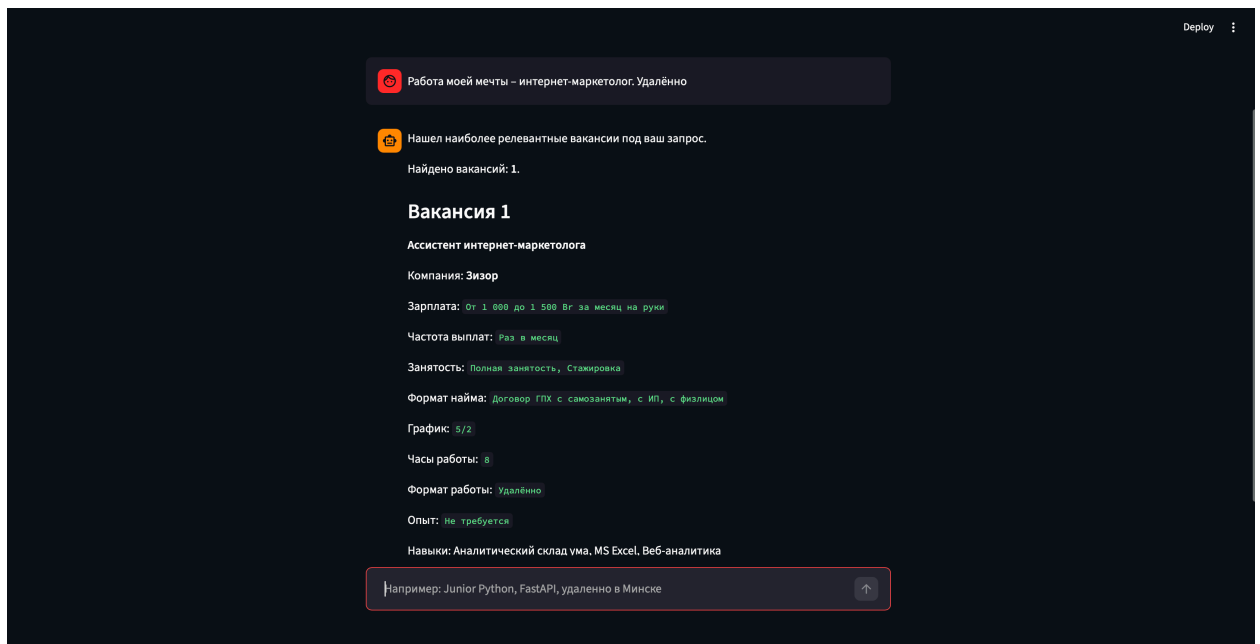


Рисунок 4.3 – Интерфейс поиска по запросу на удаленную работу интернет-маркетолога

Примечание – Источник: собственная разработка.

На рисунке 4.4 показан запрос студента на работу бариста или повара в кофейне с пожеланием частичной занятости либо графика 2/2. На первой позиции выдается вакансия «Повар в сеть кафе» с графиком 2/2, поэтому система улавливает домен horeca и часть ограничений по графику, однако не сохраняет точную роль бариста и не удовлетворяет требование частичной занятости. Следовательно, для смешанных бытовых запросов система пока лучше определяет общую предметную область, чем узкую специализацию внутри нее.

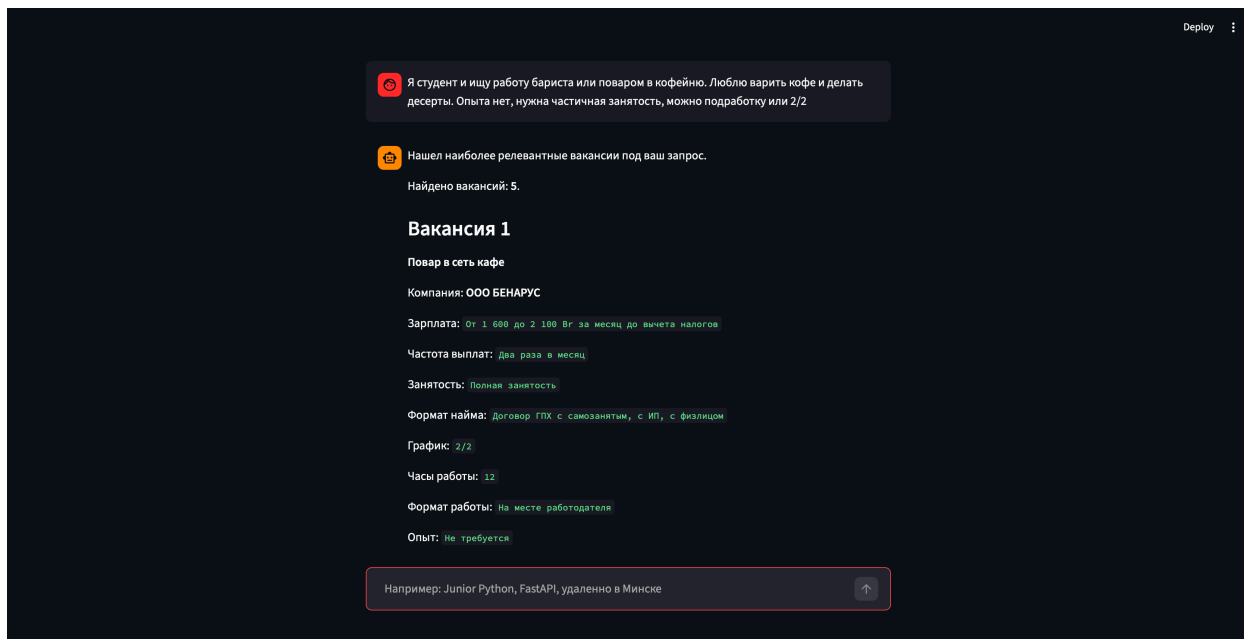


Рисунок 4.4 – Интерфейс поиска по запросу о работе в кофейне

Примечание – Источник: собственная разработка.

Таким образом, демонстрационные сценарии подтверждают практическую работоспособность интерфейса и одновременно выявляют типичные границы качества. Наиболее уверенно прототип работает в случаях, где профессиональная роль выражена явно или поддерживается характерными доменными терминами, тогда как запросы со слабо формализованными требованиями к опыту и формату занятости еще требуют дополнительной настройки.

4.3 Количественная оценка качества поиска

Контрольное испытание полного бенчмарк-набора из 16 запросов зафиксировало значения $\text{hit}@1 = 14/16$ и $\text{hit}@5 = 14/16$. Это означает, что в четырнадцати сценариях из шестнадцати релевантная вакансия попадала в первые пять результатов, и в четырнадцати сценариях из шестнадцати релевантный результат занимал первую позицию. Среднее косинусное расстояние top-1 по тем случаям, где система выдала результат, составило 0.5017. Минимальное значение наблюдалось для логистических запросов, максимальное – для сценария с запросом о ПВЗ (Ozon/Wildberries), где

релевантная вакансия также получена на первой позиции, но с наибольшим косинусным расстоянием среди рассмотренных top-1 результатов.

Ошибки ранжирования в ряде сценариев обусловлены семантической близостью различных стартовых (junior) позиций. Так, при поиске стажировки в маркетинге система предложила вакансии в сфере продаж, а запрос на роль в HR привел к выдаче IT-вакансий начального уровня. Полученные результаты указывают на ограниченную способность би-энкодера модели различать узкоспециализированные домены на начальном карьерном этапе. Это подтверждает гипотезу о необходимости внедрения дополнительного этапа реранжирования (cross-encoder) для более точной дискриминации похожих профессий.

Таблица 4.1 – Характерные результаты контрольного тестирования

Сценарий запроса	Top-1 результат	hit@1	hit@5	Комментарий
ПВЗ, Wildberries/ Ozon, работа на месте	Менеджер пункта выдачи заказов Wildberries (Лиможа)	Да	Да	Релевантная роль найдена сразу на первой позиции.
Бариста / кофейня / без опыта	Бариста "Джезва"	Да	Да	Точное совпадение по роли и домену horeca.
Support / help desk / удаленно	Support Specialist (Help Desk Analyst)	Да	Да	Точное совпадение по роли и формату работы.
HR / junior / стажировка	Intern/Junior Salesforce Developer	Нет	Нет	найдена смежная junior-роль, но не HR/подбор.

Сценарий запроса	Топ-1 результат	hit@1	hit@5	Комментарий
Ассистент интернет-маркетолога / удаленно	Junior Sales Manager company=ООО Сенлайн	Нет	Нет	Найдена смежная junior-роль, но не в сфере маркетинга
ПВЗ Ozon/ Wildberries / выдача заказов	Продавец-кассир в пункт выдачи заказов, город Гродно	Да	Да	Релевантная вакансия заняла первое место.

Окончание таблицы 4.1

Примечание – Источник: собственная разработка.

Как видно из таблицы 4.1 и полного протокола тестирования в приложении Б, сильной стороной системы являются сценарии с явно выраженной профессиональной ролью и/или устойчивыми доменными маркерами. К таким случаям относятся запросы по ПВЗ, бариста, support/help desk, программисту 1С, логистике, торговым ролям, водителям и кондитерам, где в большинстве случаев достигается корректный top-1 результат.

Наиболее проблемными в текущем прогоне остаются сценарии, связанные с разграничением смежных junior-направлений: запрос на стажировку/junior в интернет-маркетинге и запрос на стартовую роль в подборе персонала (HR). В этих случаях top-1 уходит в близкие, но нерелевантные по целевой роли вакансии, и релевантный маркер не попадает в top-5.

Таким образом, текущие ограничения связаны не с техническим срывом пайплайна, а с семантическим смешением соседних стартовых ролей. Практически это указывает на необходимость усиления финального реранжирования и более точной дискриминации между близкими junior-профилями.

4.4 Абляционный анализ: сопоставление конфигураций А и В

Для оценки вклада каскадной архитектуры выполнено сопоставление двух конфигураций. Конфигурация А представляет собой чистый bi-encoder-поиск по косинусному расстоянию без LLM-фильтров, без релаксации фильтров и без доменного реранжирования. Конфигурация В представляет полный каскад: LLM-фильтры, релаксация ограничений и domain boost. Такой дизайн позволяет сравнить базовую и полную версии системы в единой постановке.

Полный каскад в зафиксированном локальном анализе показал hit@1 равный 14/16 и hit@5 равный 14/16. Сравнение с чистым bi-encoder показывает, что надстройка полного каскада существенно повышает точность top-1 и top-5 по сравнению с чистым bi-encoder в данном прогоне.

В данной версии работы абляционный анализ ограничен сравнением конфигураций А и В. Пошаговая декомпозиция вклада отдельных модулей может быть выполнена в дальнейших экспериментах.

Таблица 4.2 – Сопоставление базовой и полной конфигураций поиска

Конфигурация	hit@1	hit@5	Интерпретация результата
А. Чистый bi-encoder без LLM, релаксации и domain boost	9/16	13/16	Хорошее покрытие top-5, но неоднозначные запросы чаще ранжируются неточно на первой позиции.
В. Полный каскад: LLM, релаксация и domain boost	14/16	14/16	Существенно повышает точность top-1 и top-5 по сравнению с чистым bi-encoder в данном прогоне.

Примечание – Источник: собственная разработка.

Таким образом, абляционный анализ подтверждает практический смысл каскадной архитектуры, но одновременно показывает, что ее качество определяется не только математикой ранжирования, а всей цепочкой: от

корректности LLM-извлечения фильтров до устойчивости доменного словаря.

Сравнение конфигураций А и В показывает, что полный каскад (В) повышает hit@1 и hit@5 относительно базовой схемы (А). Следовательно, каскадная архитектура в формате «базовая модель против полного pipeline» дает измеримый прирост качества поиска.

4.5 Визуализация процесса принятия решения

Логика основного сценария пользовательского поиска может быть представлена в виде UML activity-диаграммы. Такая визуализация позволяет увидеть, что система представляет собой не один вызов векторного поиска, а последовательность связанных стадий с развилками, циклами повторной валидации и этапом смягчения фильтров. Для экспериментального анализа это важно, поскольку точки возникновения ошибок и точки улучшения качества находятся на разных шагах pipeline.

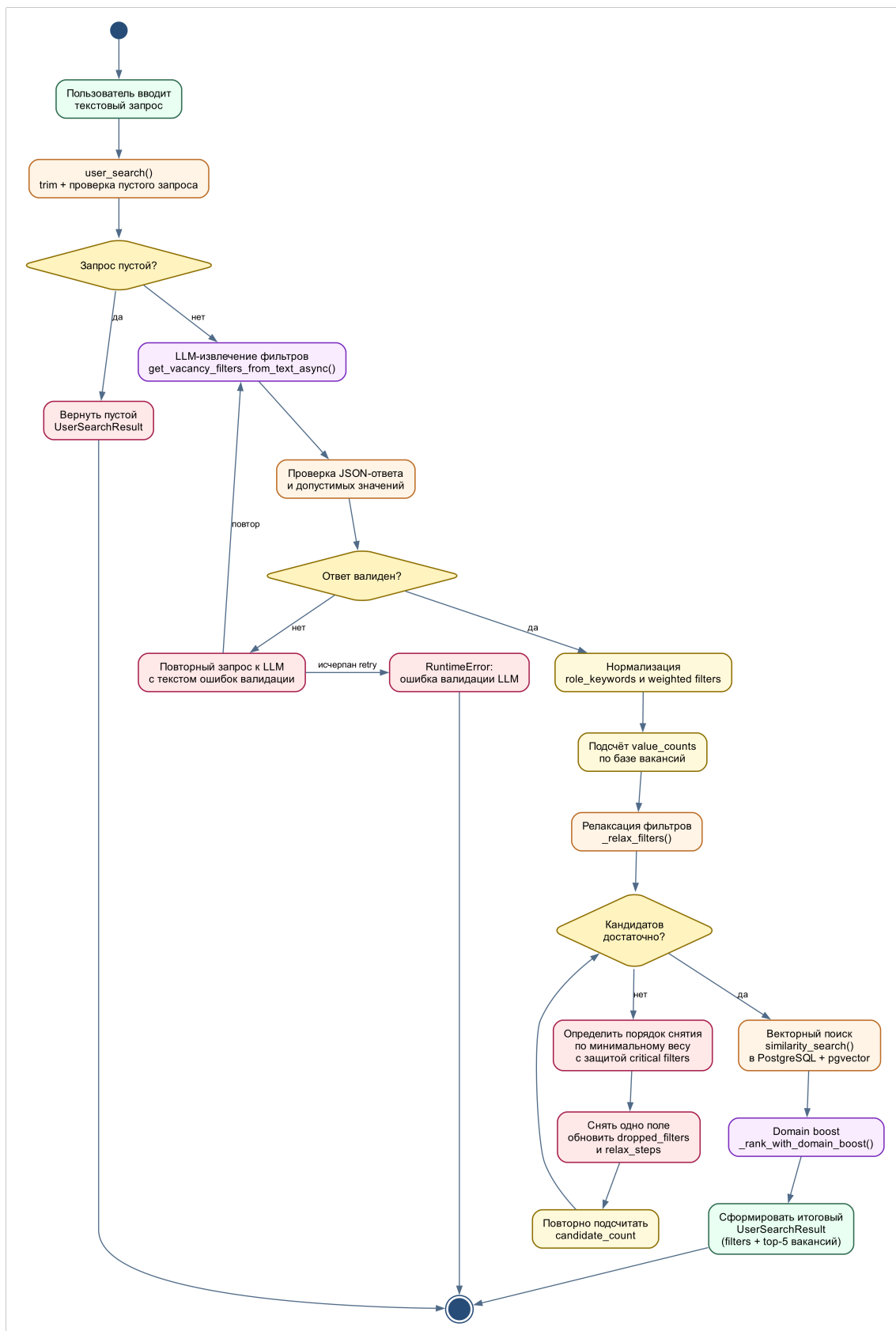


Рисунок 4.5 – UML activity-диаграмма сценария пользовательского поиска

Примечание – Источник: собственная разработка.

На рисунке 4.5 показано, что после ввода запроса система сначала проверяет его корректность и обращается к LLM-слою для получения структурированных фильтров. Далее на основе реального распределения значений в базе оценивается число потенциальных кандидатов. Если ограничений слишком много и выборка становится чрезмерно узкой, активируется цикл релаксации: удаляются наименее важные фильтры, после чего кандидатное множество пересчитывается повторно.

После достижения допустимого числа кандидатов выполняется векторный поиск по полю *embedding*, затем результаты проходят этап *domain boost*, где учитываются совпадения по профессиональным и доменным ключевым словам. На выходе формируется *top-5* вакансий вместе с набором метаданных: примененными фильтрами, снятыми ограничениями и шагами релаксации. Тем самым *activity*-диаграмма фиксирует как путь успешного завершения сценария, так и альтернативную ветвь завершения с ошибкой при неудачном LLM-ответе.

4.6 Выводы по главе 4

1. На наборе из 16 бенчмарк-запросов полный каскад показал полезное качество поиска: *hit@1* равен 14/16, а *hit@5* – 14/16, что подтверждает работоспособность выбранной архитектуры в прикладном режиме.
2. Наиболее устойчиво система работает в сценариях с явно выраженными профессиональными ролями и доменными маркерами: ПВЗ, *support/help desk*, программист 1С, логистика, торговые роли, водительские вакансии, кондитерские и *horeca*-сценарии.
3. Основные проблемные зоны в текущем прогоне связаны с двумя сценариями из 16, где релевантный маркер не попадает в *top-5*: (а) запрос на стажировку/*junior* в интернет-маркетинге и (б) запрос на стартовую HR-роль (подбор персонала). В обоих случаях *top-1* смещается к смежным *junior*-позициям.
4. Сопоставление с чистым *bi-encoder*-поиском показало, что полный каскад существенно повышает точность *top-1* и *top-5* в данном прогоне, поэтому дальнейшее развитие целесообразно строить вокруг более точной дискриминации близких *junior*-профилей и усиления финального реранжирования.

5 НАПРАВЛЕНИЯ ДАЛЬНЕЙШЕГО РАЗВИТИЯ

5.1 Усиление моделей retrieval и ранжирования

Результаты экспериментальной оценки, приведенные в четвертой главе, показывают, что основное ограничение текущего прототипа связано не столько с отсутствием релевантных вакансий в корпусе, сколько с недостаточной точностью их расположения в верхней части выдачи. Особенно это заметно для коротких и неоднозначных запросов, в которых близкие по домену роли, например бариста и кассир либо сотрудник ПВЗ и мерчендайзер, оказываются семантически соседними. Следовательно, первоочередным направлением развития следует считать усиление именно модели финального ранжирования.

Перспективным решением является переход от одноступенчатого поиска к двухступенчатой схеме. На первом этапе существующая bi-encoder модель может по-прежнему быстро отбирать ограниченное число кандидатов по эмбедингам, а на втором этапе более точная модель попарного сопоставления запроса и вакансии будет уточнять порядок элементов внутри top-k. Подобный reranking особенно важен для запросов соискателей, где формулировка намерения часто оказывается бытовой и неполной, а решение должно учитывать тонкие различия между смежными ролями.

Дополнительно целесообразно развивать гибридный retrieval-контур, объединяющий векторное сопоставление с лексическими признаками по полям title и skills. Для рассматриваемой предметной области это позволит увереннее обрабатывать короткие профессиональные маркеры и специальные обозначения, такие как 1C, HR, Ozon, Wildberries, junior или help desk, которые не всегда достаточно устойчиво проявляются в плотном семантическом пространстве. Еще одним направлением является доменная адаптация текущей MiniLM-модели на парах «запрос - релевантная вакансия» и «запрос - нерелевантная вакансия», что позволит заменить фиксированную эвристику взвешивания полей более обоснованной обучаемой схемой.

5.2 Развитие модели интерпретации запроса и генеративного контура

В текущей версии системы языковая модель используется главным образом для извлечения структурированных фильтров и `role_keywords` из естественного языка. Такой подход уже повышает гибкость интерфейса, однако результаты главы 4 показали его зависимость от валидности внешнего LLM-ответа. Поэтому следующим логичным шагом должно стать построение более устойчивой модели интерпретации запроса, в которой детерминированная нормализация типовых значений сочетается с генеративным анализом только для неоднозначных случаев.

Практически это может быть реализовано как двухуровневый контур: на первом уровне правила, словари синонимов и онтология ролей выделяют наиболее частые ограничения по опыту, формату занятости и графику, а на втором уровне LLM доуточняет намерение пользователя, если запрос содержит сложные или смешанные формулировки. Подобная схема уменьшит число ошибок валидации, повысит воспроизводимость экспериментов и снизит стоимость одного поискового сценария. Данное направление особенно значимо, поскольку фактически речь идет о переходе от одной универсальной модели к композиции правилковой и вероятностной моделей принятия решения.

Отдельным направлением развития выступает расширение генеративного слоя после этапа `retrieval`. В проект может быть добавлена идея `endpoint`-маршрута анализа, который мог бы формировать краткое `summary` для каждой найденной вакансии, объяснение причин ее релевантности, сильные и слабые стороны предложения, а также краткое сопоставление нескольких позиций между собой. В дальнейшем целесообразно добавить и структурированную память пользовательской сессии, чтобы система сохраняла контекст предыдущих уточнений запроса и поддерживала итеративный диалоговый поиск, а не только одноразовый запрос без истории.

Важным этапом развития является замыкание цикла RAG. Это подразумевает внедрение генеративного слоя, ответственного за формирование персональных текстовых отчетов.

5.3 Масштабирование корпуса вакансий и программной архитектуры

Существенным резервом повышения качества остается расширение базы знаний. В текущем варианте прототип опирается преимущественно на вакансии одного источника и одного города, поэтому часть пользовательских

сценариев ограничивается не только моделью ранжирования, но и фактической полнотой корпуса. В дальнейшем целесообразно организовать регулярный сбор вакансий с нескольких площадок, а также расширить географию поиска за пределы Гродно. Это сделает систему ближе к реальному рынку труда Республики Беларусь и позволит оценивать модель не на локальном, а на более репрезентативном наборе данных.

Рост корпуса неизбежно требует дальнейшего развития инфраструктуры поиска. Для ускорения работы с большими массивами вакансий может быть внедрен ANN-индекс HNSW в pgvector, а сам процесс наполнения базы следует перевести в режим периодического ge-crawl с дедупликацией, контролем качества и автоматическим пересчетом эмбедингов только для изменившихся записей. Важным дополнением станет и регулярный мониторинг качества через повторяемый бенчмарк-набор, поскольку для интеллектуальной системы недостаточно лишь поддерживать работоспособность сервиса; необходимо количественно отслеживать, как изменение каждой подмодели влияет на итоговые hit@1 и hit@5.

На уровне программной оболочки возможна и дальнейшая продуктивизация решения. Демонстрационный стек Python, FastAPI и Streamlit хорошо подходит для учебного прототипирования, однако при переходе к более сложному пользовательскому интерфейсу допустимо выделить отдельный веб-клиент на React и сервисный слой на NestJS. Вместе с тем такой шаг следует рассматривать как вторичный по отношению к развитию моделей поиска и интерпретации: без повышения качества retrieval, reranking и работы с контекстом технологическая смена фронтенда и backend-обязки сама по себе не устраним выявленные ограничения интеллектуального ядра системы.

5.4 Выводы по главе 5

1. Приоритетным направлением дальнейшего развития является повышение точности верхней части выдачи за счет гибридного retrieval-поиска, доменной адаптации эмбедингов и введения более точного этапа реранжирования кандидатов.
2. Перспективным является переход от изолированного LLM-извлечения фильтров к комбинированной модели интерпретации запроса, объединяющей правила, словари, онтологию ролей и генеративный анализ неоднозначных формулировок.

3. Развитие генеративного контура целесообразно связывать с построением summary для каждой вакансии, объяснением причин релевантности и поддержкой сессионной памяти, необходимой для многошагового диалогового поиска.
4. Масштабирование системы должно включать расширение корпуса вакансий, многоисточниковый сбор данных, ускорение поиска на больших массивах и лишь после стабилизации интеллектуального ядра - последующую продуктивизацию пользовательской архитектуры.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы была решена задача разработки прототипа интеллектуальной системы семантического поиска вакансий, ориентированной на рынок труда Республики Беларусь. Полученные результаты позволяют сделать следующие выводы.

1. В параграфах 1.1 и 1.2 систематизированы основные подходы к решению задач информационного поиска и ранжирования, а также показано, что задача подбора вакансий может рассматриваться как частный случай интеллектуального поиска по корпусу документов. Это позволило обоснованно выбрать класс bi-encoder моделей Sentence-BERT как основу для последующей реализации семантического сопоставления запросов и вакансий.
2. В параграфах 1.3 и 1.4 обосновано использование косинусной меры близости для нормированных эмбедингов и показано, что архитектура Retrieval Augmented Generation является естественной теоретической рамкой для построения системы, объединяющей retrieval компонент, языковую модель и внешний корпус актуальных вакансий. Тем самым сформирована теоретическая база для выбора методов и инструментальных средств проекта.
3. В параграфах 2.1 и 2.2 выявлены ключевые особенности рынка труда Республики Беларусь и ограничения существующих HR платформ. Установлено, что для рассматриваемой предметной области существенны мультязычность, вариативность названий ролей, неполнота пользовательских формулировок и проблема нулевой выдачи при слишком жесткой фильтрации.
4. В параграфе 2.3 сформулирована постановка задачи, определены функциональные и нефункциональные требования к системе и зафиксированы количественные ориентиры результата. В частности, заданы требования к формированию top five выдачи, к использованию эмбедингов размерности 384 и к достижению практически полезного качества поиска на эталонном наборе пользовательских запросов.
5. В параграфах 2.4, 3.1 и 3.2 обоснован выбор технологического стека Python, FastAPI, Streamlit, PostgreSQL и pgvector, а также спроектирована трехуровневая архитектура системы. Реализованная модель хранения знаний объединяет структурированные поля вакансии и векторное представление vector(384), что обеспечивает единое пространство для фильтрации, подсчета кандидатов и семантического ранжирования.

6. В параграфе 3.3 реализован контур автоматизированного сбора вакансий с портала rabota.by, включающий инкрементную обработку данных, нормализацию карточек, устойчивость к сетевым сбоям и пошаговое сохранение результатов. В текущей версии проекта на этой основе сформирован корпус объемом 1016 вакансий, пригодный как для прикладного поиска, так и для воспроизводимой экспериментальной оценки.
7. В параграфах 3.4, 3.5, 3.6 и 3.7 реализован основной каскад поисковых моделей. Его ключевыми компонентами стали авторская схема взвешенного текстового представления вакансии, LLM извлечение структурированных фильтров с весами важности, жадная релаксация наименее значимых ограничений и доменное реранжирование результатов, что позволило повысить смысловую точность выдачи и снизить риск пустого ответа пользователю.
8. В параграфах 4.1, 4.2, 4.3 и 4.4 выполнена экспериментальная проверка полного pipeline на наборе из 16 бенчмарк запросов. Установлено, что полный каскад обеспечивает значение $\text{hit}@1$ равное 14 из 16 и значение $\text{hit}@5$ равное 14 из 16, а сопоставление с базовой конфигурацией без интеллектуальной надстройки показывает выигрыш прежде всего по точности первой позиции.
9. Проведенное исследование позволило выделить основные проблемные зоны текущей реализации. В финальном контрольном прогоне два сценария из шестнадцати не достигли попадания релевантного маркера в top-5: (а) запрос на стажировку/junior в интернет-маркетинге; (б) запрос на стартовую роль в подборе персонала (HR). В обоих случаях top-1 смещается к смежным junior-позициям, что указывает на необходимость более точной дискриминации близких стартовых профилей на этапе финального реранжирования.
10. В главе 5 определены пути дальнейшего совершенствования разработанного решения. Наиболее перспективными из них являются переход к двухступенчатому retrieval сценарию с более точным reranker компонентом, построение комбинированной модели интерпретации запроса на основе правил и языковой модели, расширение корпуса вакансий, внедрение ANN индекса HNSW, а также развитие генеративного слоя для объяснения релевантности найденных результатов и поддержки многошагового диалога с пользователем.

Таким образом, цель курсовой работы достигнута, поставленные задачи выполнены, а разработанный прототип подтвердил практическую состоятельность подхода к семантическому поиску вакансий на основе каскада взаимосвязанных моделей.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Мэннинг К. Д., Рагхаван П., Шютце Х. Введение в информационный поиск. – М. : Вильямс, 2011. – 528 с.
2. Нгуен Б. Н., Тузовский А. Ф. Обзор подходов семантического поиска // Доклады ТУСУР. – 2010. – № 2(22), ч. 2. – С. 234–237.
3. Лейн Х., Хапке Х. Обработка естественного языка в действии. – СПб. : Питер, 2020. – 576 с.
4. Жерон О. Прикладное машинное обучение с помощью Scikit-Learn, Keras и TensorFlow: концепции, инструменты и техники для создания интеллектуальных систем. – 2-е изд. – М. : Диалектика, 2020. – 1039 с.
5. Гудфеллоу Я., Бенджио И., Курвилль А. Глубокое обучение. – М. : ДМК Пресс, 2017. – 652 с.
6. Ботов Д. С., Кленин Ю. Д., Николаев И. Е. Извлечение информации с использованием нейросетевых моделей языка на примере анализа вакансий в системах онлайн-рекрутмента // Вестник Югорского государственного университета. – 2018. – Вып. 3(50). – С. 37–48.
7. Ботов Д. С., Дмитриин Ю. В., Кленин Ю. Д. Семантический поиск учебных дисциплин под требования рынка труда на основе нейросетевых моделей языка // Вестник ЮУрГУ. Серия «Компьютерные технологии, управление, радиоэлектроника». – 2019. – Т. 19, № 2. – С. 5–15.
8. Ротман Д. RAG и генеративный ИИ. Создаем собственные RAG-пайплайны с помощью LlamaIndex, Deep Lake и Pinecone. – СПб. : Питер, 2025. – 320 с.
9. RAG (Retrieval-Augmented Generation): основы и продвинутые техники [Электронный ресурс] / Хабр. – Режим доступа: <https://habr.com/ru/articles/871226/>. – Дата доступа: 26.04.2026.
10. Семантический поиск: от простого сходства Жаккара к сложному SBERT [Электронный ресурс] / Хабр. – Режим доступа: <https://habr.com/ru/companies/skillfactory/articles/566414/>. – Дата доступа: 26.04.2026.
11. FastAPI [Электронный ресурс] / официальная документация. – Режим доступа: <https://fastapi.tiangolo.com/ru/>. – Дата доступа: 26.04.2026.
12. Ричардс Т., Груздев А. Streamlit для Data Science. Создаем интерактивные приложения в Python. – М. : ДМК Пресс, 2024. – 354 с.
13. Документация к PostgreSQL 18.3 [Электронный ресурс] / Postgres Professional. – Режим доступа: <https://postgrespro.ru/docs/postgresql/current>. – Дата доступа: 26.04.2026.

14. Векторный поиск внутри PostgreSQL: что умеет и где может пригодиться pgvector [Электронный ресурс] / А. Гришин. – Режим доступа: <https://habr.com/ru/companies/selectel/articles/920824/>. – Дата доступа: 26.04.2026.

15. Векторный поиск в PostgreSQL: PGVector, SQLAlchemy и FastAPI [Электронный ресурс] / Хабр. – Режим доступа: <https://habr.com/ru/articles/1006364/>. – Дата доступа: 26.04.2026.

16. Готовим SQLAlchemy правильно [Электронный ресурс] / Recovery Mode // Хабр. – Режим доступа: <https://habr.com/ru/articles/851472/>. – Дата доступа: 26.04.2026.

17. OpenRouter: единый API для доступа к LLM [Электронный ресурс] / Хабр. – Режим доступа: <https://habr.com/ru/articles/943186/>. – Дата доступа: 26.04.2026.

18. Sentence Transformers: paraphrase-multilingual-MiniLM-L12-v2 [Электронный ресурс] / Hugging Face. – Режим доступа: <https://huggingface.co/sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2>. – Дата доступа: 26.04.2026.

19. rabota.by [Электронный ресурс] / сервис поиска работы и подбора персонала. – Режим доступа: <https://rabota.by/>. – Дата доступа: 26.04.2026.

ПРИЛОЖЕНИЕ А

Листинги ключевых модулей

```
"""Compute and persist vacancy embeddings (weighted structured
fields; description optional)."""

from __future__ import annotations

import asyncio
from typing import Any

from sqlalchemy import select, update
from sqlalchemy.ext.asyncio import AsyncSession

from common.constants.embedding import (
    DEFAULT_EMBED_BATCH_SIZE,
    VACANCY_EMBED_COMPANY_REPEATS,
    VACANCY_EMBED_DESCRIPTION_MAX_CHARS,
    VACANCY_EMBED_INCLUDE_DESCRIPTION,
    VACANCY_EMBED_SKILLS_REPEATS,
    VACANCY_EMBED_STRUCTURE_REPEATS,
    VACANCY_EMBED_TITLE_REPEATS,
)
from features.database.db import async_session_factory
from features.database.models import Vacancy
from features.embedding.encoder import encode_texts,
require_pgvector_embedding

_SKIP_PLACEHOLDERS: frozenset[str] = frozenset(
    {"", "не указано", "не указаны", "-"},
)

_STRUCTURE_FIELDS: tuple[tuple[str, str], ...] = (
    ("Занятость", "employment"),
    ("График", "schedule"),
    ("Часы", "hours"),
    ("Формат работы", "work_format"),
    ("Опыт", "experience"),
    ("Трудоустройство", "hiring_format"),
)

def _trim(s: str) -> str:
    return s.strip()

def _skip_value(value: str) -> bool:
    t = _trim(value).lower()
    return not t or t in _SKIP_PLACEHOLDERS

def _repeat_lines(label: str, value: str, times: int) ->
list[str]:
    if _skip_value(value):
        return []
    line = f"{label}: {_trim(value)}"
    return [line] * times
```

```

def _description_for_embed(raw: str) -> str | None:
    """Return description slice for embedding, or None if
    disabled / empty."""
    if not VACANCY_EMBED_INCLUDE_DESCRIPTION:
        return None
    t = _trim(raw)
    if not t:
        return None
    cap = VACANCY_EMBED_DESCRIPTION_MAX_CHARS
    if cap is not None and len(t) > cap:
        return t[:cap].rstrip() + "..."
    return t

def _vacancy_embed_text(row: Any) -> str:
    """
    Build one string for embedding: title and structured fields
    (weighted by repetition),
    then skills. Description is optional (see
    ``VACANCY_EMBED_INCLUDE_DESCRIPTION``).
    """
    chunks: list[str] = []

    title = _trim(row.title)
    if title:
        job_line = f"Должность: {title}"
        chunks.extend([job_line] * VACANCY_EMBED_TITLE_REPEATS)

    chunks.extend(_repeat_lines("Компания", row.company,
        VACANCY_EMBED_COMPANY_REPEATS))
    for label, attr in _STRUCTURE_FIELDS:
        chunks.extend(
            _repeat_lines(
                label,
                getattr(row, attr),
                VACANCY_EMBED_STRUCTURE_REPEATS,
            )
        )
    chunks.extend(_repeat_lines("Зарплата", row.salary, 1))
    chunks.extend(_repeat_lines("Выплаты",
        row.payment_frequency, 1))

    if not _skip_value(row.skills):
        sk = f"Навыки: {_trim(row.skills)}"
        chunks.extend([sk] * VACANCY_EMBED_SKILLS_REPEATS)

    desc = _description_for_embed(row.description)
    if desc:
        chunks.append(f"Описание: {desc}")

    return "\n\n".join(chunks)

async def _fetch_batch(
    session: AsyncSession,
    *,
    after_id: int,
    batch_size: int,

```

```

        force: bool,
    ) -> list[Any]:
        stmt = (
            select(
                Vacancy.id,
                Vacancy.title,
                Vacancy.company,
                Vacancy.salary,
                Vacancy.payment_frequency,
                Vacancy.experience,
                Vacancy.employment,
                Vacancy.hiring_format,
                Vacancy.schedule,
                Vacancy.hours,
                Vacancy.work_format,
                Vacancy.skills,
                Vacancy.description,
            )
            .where(Vacancy.id > after_id)
            .order_by(Vacancy.id)
            .limit(batch_size)
        )
        if not force:
            stmt = stmt.where(Vacancy.embedding.is_(None))
        result = await session.execute(stmt)
        return list(result.all())

async def _persist_vectors(
    session: AsyncSession,
    ids: list[int],
    vectors: list[list[float]],
) -> None:
    for vid, vec in zip(ids, vectors, strict=True):
        await session.execute(
            update(Vacancy).where(Vacancy.id ==
vid).values(embedding=vec)
        )
    await session.commit()

async def _embed_and_store_batch(
    session: AsyncSession,
    rows: list[Any],
) -> int:
    texts = [_vacancy_embed_text(r) for r in rows]
    vectors = await encode_texts(texts)
    ids = [r.id for r in rows]
    await _persist_vectors(session, ids, vectors)
    return len(ids)

async def fill_db_with_vectors(
    *,
    batch_size: int = DEFAULT_EMBED_BATCH_SIZE,
    force: bool = False,
) -> int:
    """
    Fill ``vacancies.embedding`` using Sentence-BERT
    (multilingual MiniLM, dim 384).

```

```

    Text combines weighted structured fields (title, schedule,
    employment, ...), skills,
    and optionally the full description (see
    ``VACANCY_EMBED_INCLUDE_DESCRIPTION``). After
    changing embedding settings in
    ``common.constants.embedding``, run with ``force=True``
    to refresh all vectors.

    By default updates only rows with ``embedding IS NULL``. Set
    ``force=True`` to
    recompute embeddings for every vacancy (keyset pagination by
    ``id``).

    Returns the number of rows updated.
    """
    require_pgvector_embedding()
    total = 0
    after_id = 0
    async with async_session_factory() as session:
        while True:
            rows = await _fetch_batch(
                session,
                after_id=after_id,
                batch_size=batch_size,
                force=force,
            )
            if not rows:
                break
            after_id = rows[-1].id
            total += await _embed_and_store_batch(session, rows)
    return total

def run_fill_db_with_vectors(
    *,
    batch_size: int = DEFAULT_EMBED_BATCH_SIZE,
    force: bool = False,
) -> int:
    """
    Run :func:`fill_db_with_vectors` from synchronous code (CLI,
    scripts).

    Wraps ``asyncio.run`` so callers do not need to manage an
    event loop.
    """
    return
    asyncio.run(fill_db_with_vectors(batch_size=batch_size,
    force=force))

```

Листинг А.1 – Модуль взвешенной векторизации вакансий

Примечание – Источник: собственная разработка.

```

"""Semantic search over ``vacancies.embedding`` using pgvector
cosine distance."""

```

```

from __future__ import annotations

import asyncio

from sqlalchemy import or_, select
from sqlalchemy.ext.asyncio import AsyncSession

from common.constants.embedding import DEFAULT_SIMILARITY_TOP_K
from common.constants.llm import VACANCY_FILTER_KEYS
from features.database.db import async_session_factory
from features.database.models import Vacancy
from features.embedding.encoder import encode_texts,
require_pgvector_embedding
from features.embedding.schemas import SimilaritySearchResult

_FILTER_COLUMN_MAP = {
    "payment_frequency": Vacancy.payment_frequency,
    "experience": Vacancy.experience,
    "employment": Vacancy.employment,
    "hiring_format": Vacancy.hiring_format,
    "schedule": Vacancy.schedule,
    "hours": Vacancy.hours,
    "work_format": Vacancy.work_format,
}

def vacancy_to_result(row: Vacancy, distance: float) ->
SimilaritySearchResult:
    return SimilaritySearchResult(
        id=row.id,
        title=row.title,
        company=row.company,
        salary=row.salary,
        payment_frequency=row.payment_frequency,
        experience=row.experience,
        employment=row.employment,
        hiring_format=row.hiring_format,
        schedule=row.schedule,
        hours=row.hours,
        work_format=row.work_format,
        skills=row.skills,
        url=row.url,
        description=row.description,
        cosine_distance=float(distance),
    )

def normalize_filters(
    filters: dict[str, list[str]] | None,
) -> dict[str, list[str]]:
    if not filters:
        return {}
    unknown = sorted(set(filters) - set(VACANCY_FILTER_KEYS))
    if unknown:
        msg = f"Unknown filter keys: {' '.join(unknown)}."
        raise ValueError(msg)
    normalized: dict[str, list[str]] = {}
    for key in VACANCY_FILTER_KEYS:
        values = filters.get(key, [])

```

```

        cleaned = sorted({item.strip() for item in values if
item.strip()})
        if cleaned:
            normalized[key] = cleaned
    return normalized

def _apply_filters(
    stmt: object,
    filters: dict[str, list[str]],
):
    for key, values in filters.items():
        stmt = stmt.where(_FILTER_COLUMN_MAP[key].in_(values))
    return stmt

def _normalize_role_keywords(role_keywords: list[str] | None) ->
list[str]:
    if not role_keywords:
        return []
    normalized = sorted(
        {item.strip().lower() for item in role_keywords if
item.strip()})
    )
    return normalized[:5]

def _apply_role_keywords(stmt: object, role_keywords:
list[str]):
    if not role_keywords:
        return stmt
    clauses = [
        or_(
            Vacancy.title.ilike(f"%{keyword}%"),
            Vacancy.skills.ilike(f"%{keyword}%"),
        )
        for keyword in role_keywords
    ]
    return stmt.where(or_(*clauses))

async def _search_with_session(
    session: AsyncSession,
    query_vector: list[float],
    *,
    limit: int,
    filters: dict[str, list[str]],
    role_keywords: list[str],
) -> list[SimilaritySearchResult]:
    distance_expr =
Vacancy.embedding.cosine_distance(query_vector)
    stmt = (
        select(Vacancy, distance_expr.label("dist"))
        .where(Vacancy.embedding.isnot(None))
        .order_by(distance_expr)
        .limit(limit)
    )
    stmt = _apply_filters(stmt, filters)
    stmt = _apply_role_keywords(stmt, role_keywords)
    result = await session.execute(stmt)

```

```

        return [_vacancy_to_result(v, d) for v, d in result.all()]

async def similarity_search(
    query_text: str,
    *,
    limit: int = DEFAULT_SIMILARITY_TOP_K,
    filters: dict[str, list[str]] | None = None,
    role_keywords: list[str] | None = None,
    session: AsyncSession | None = None,
) -> list[SimilaritySearchResult]:
    """
    Vectorize ``query_text`` and return vacancies ordered by
    cosine distance (closest first).

    Uses the same Sentence-BERT model and dimension (384)
    as :mod:`save_vectors_service`.
    """
    require_pgvector_embedding()
    stripped = query_text.strip()
    if not stripped:
        return []

    vectors = await encode_texts([stripped])
    query_vector = vectors[0]
    normalized_filters = _normalize_filters(filters)
    normalized_role_keywords =
    _normalize_role_keywords(role_keywords)

    if session is not None:
        return await _search_with_session(
            session,
            query_vector,
            limit=limit,
            filters=normalized_filters,
            role_keywords=normalized_role_keywords,
        )

    async with async_session_factory() as s:
        return await _search_with_session(
            s,
            query_vector,
            limit=limit,
            filters=normalized_filters,
            role_keywords=normalized_role_keywords,
        )

def run_similarity_search(
    query_text: str,
    *,
    limit: int = DEFAULT_SIMILARITY_TOP_K,
    filters: dict[str, list[str]] | None = None,
    role_keywords: list[str] | None = None,
) -> list[SimilaritySearchResult]:
    """Synchronous wrapper for :func:`similarity_search`
    (scripts, REPL)."""
    return asyncio.run(
        similarity_search(
            query_text,

```

```

        limit=limit,
        filters=filters,
        role_keywords=role_keywords,
    )
)

```

Листинг A.2 – Модуль векторного поиска вакансий через pgvector

Примечание – Источник: собственная разработка.

```

"""OpenRouter service helpers for extracting vacancy filters
from user text."""

from __future__ import annotations

import asyncio
import json
import os
import time
from typing import Any

import requests
from sqlalchemy import func, select

from common.constants.llm import (
    LLM_VALIDATION_RETRY_ATTEMPTS,
    VACANCY_OPTIONAL_KEYS,
    OPENROUTER_BASE_URL,
    OPENROUTER_MODEL,
    OPENROUTER_RETRY_ATTEMPTS,
    OPENROUTER_RETRY_BACKOFF_SECONDS,
    OPENROUTER_TIMEOUT_SECONDS,
    VACANCY_FILTER_KEYS,
    VACANCY_FILTERS_CONTEXT_TEMPLATE,
    VACANCY_FILTERS_SYSTEM_PROMPT_TEMPLATE,
)
from features.database.db import async_session_factory
from features.database.models import Vacancy
from features.llm.schemas import LlmAnswer

_FILTER_COLUMN_MAP = {
    "payment_frequency": Vacancy.payment_frequency,
    "experience": Vacancy.experience,
    "employment": Vacancy.employment,
    "hiring_format": Vacancy.hiring_format,
    "schedule": Vacancy.schedule,
    "hours": Vacancy.hours,
    "work_format": Vacancy.work_format,
}

def _get_openrouter_api_key() -> str:
    api_key = os.getenv("OPEN_ROUTER_API_KEY")
    if api_key and api_key.strip():
        return api_key.strip()
    msg = "OPEN_ROUTER_API_KEY is missing. Add it to environment
variables."

```

```

raise ValueError(msg)

def _build_payload(prompt: str, context: str) -> dict[str, Any]:
    return {
        "model": OPENROUTER_MODEL,
        "temperature": 0,
        "messages": [
            {"role": "system", "content": prompt},
            {"role": "user", "content": context},
        ],
    }

def _extract_content(data: dict[str, Any]) -> str:
    choices = data.get("choices")
    if not isinstance(choices, list) or not choices:
        raise ValueError("OpenRouter response does not contain choices.")
    message = choices[0].get("message")
    if not isinstance(message, dict):
        raise ValueError("OpenRouter response message is missing.")
    content = message.get("content")
    if not isinstance(content, str) or not content.strip():
        raise ValueError("OpenRouter response content is empty.")
    return content.strip()

def _strip_code_fences(text: str) -> str:
    cleaned = text.strip()
    if cleaned.startswith("`"):
        cleaned = cleaned[3:]
        if cleaned.startswith("json"):
            cleaned = cleaned[4:]
        cleaned = cleaned.strip()
        if cleaned.endswith("`"):
            cleaned = cleaned[:-3]
    return cleaned.strip()

def _parse_json_payload(raw_content: str) -> dict[str, Any]:
    normalized = _strip_code_fences(raw_content)
    parsed = json.loads(normalized)
    if not isinstance(parsed, dict):
        raise ValueError("LLM output must be a JSON object.")
    return parsed

def _empty_filter_object() -> dict[str, list[str]]:
    return {key: [] for key in VACANCY_FILTER_KEYS}

def _empty_filter_count_object() -> dict[str, dict[str, int]]:
    return {key: {} for key in VACANCY_FILTER_KEYS}

def _build_system_prompt(allowed_values: dict[str, list[str]])
-> str:

```

```

        allowed_values_json = json.dumps(allowed_values,
ensure_ascii=False)
        return VACANCY_FILTERS_SYSTEM_PROMPT_TEMPLATE.format(
            allowed_values_json=allowed_values_json
        )

def _normalize_db_value(value: str | None) -> str | None:
    if value is None:
        return None
    normalized = value.strip()
    return normalized or None

def _validate_llm_payload(
    payload: dict[str, Any], allowed_values: dict[str,
list[str]]
) -> list[str]:
    errors: list[str] = []
    required = set(VACANCY_FILTER_KEYS)
    optional = set(VACANCY_OPTIONAL_KEYS)
    payload_keys = set(payload.keys())
    if not required.issubset(payload_keys):
        errors.append("JSON must contain all required filter
keys.")
    unknown = sorted(payload_keys - required - optional)
    if unknown:
        errors.append(f"JSON contains unknown keys: {'',
'.join(unknown)}".)

    role_keywords = payload.get("role_keywords", [])
    if not isinstance(role_keywords, list):
        errors.append("Field 'role_keywords' must be an array.")
    elif not all(isinstance(item, str) for item in
role_keywords):
        errors.append("Field 'role_keywords' must contain only
strings.")
    elif len(role_keywords) > 5:
        errors.append("Field 'role_keywords' must contain at
most 5 items.")
    for key in VACANCY_FILTER_KEYS:
        value = payload.get(key)
        if not isinstance(value, list):
            errors.append(f"Field '{key}' must be an array.")
            continue
        allowed_set = set(allowed_values[key])
        invalid_values: list[str] = []
        for item in value:
            if not isinstance(item, dict):
                errors.append(f"Field '{key}' items must be
objects.")
                continue
            if set(item.keys()) != {"value", "weight"}:
                errors.append(
                    f"Field '{key}' items must contain only
'value' and 'weight'."
                )
                continue
            raw_filter_value = item.get("value")
            raw_weight = item.get("weight")

```

```

        if not isinstance(raw_filter_value, str):
            errors.append(f"Field '{key}' item 'value' must
be a string.")
            continue
        if not isinstance(raw_weight, int | float):
            errors.append(f"Field '{key}' item 'weight' must
be numeric.")
            continue
        if not 0 <= float(raw_weight) <= 1:
            errors.append(f"Field '{key}' item 'weight' must
be in range [0, 1].")
            continue
        if raw_filter_value not in allowed_set:
            invalid_values.append(raw_filter_value)
    if invalid_values:
        joined = ", ".join(invalid_values)
        errors.append(f"Field '{key}' has invalid values:
{joined}.")
    return errors

```

```

def _build_retry_context(base_context: str, errors: list[str])
-> str:
    validation_errors = "\n".join(f"- {err}" for err in
errors[:12])
    return (
        f"{base_context}\n\n"
        "Предыдущий ответ не прошел валидацию. "
        "Верни исправленный JSON по правилам.\n"
        f"Ошибки валидации:\n{validation_errors}"
    )

```

```

def _is_retryable_status(status_code: int) -> bool:
    return status_code == 429 or status_code >= 500

```

```

def _response_error_message(response: requests.Response) -> str:
    try:
        data = response.json()
        if isinstance(data, dict):
            error = data.get("error")
            if isinstance(error, dict) and
isinstance(error.get("message"), str):
                return error["message"]
            if isinstance(error, str):
                return error
    except ValueError:
        pass
    return response.text.strip() or "Unknown OpenRouter error."

```

```

def _post_with_retry(
    headers: dict[str, str], payload: dict[str, Any]
) -> requests.Response:
    last_error: Exception | None = None
    for attempt in range(1, OPENROUTER_RETRY_ATTEMPTS + 1):
        try:
            response = requests.post(
                OPENROUTER_BASE_URL,

```

```

        headers=headers,
        json=payload,
        timeout=OPENROUTER_TIMEOUT_SECONDS,
    )
except requests.RequestException as err:
    last_error = err
    if attempt == OPENROUTER_RETRY_ATTEMPTS:
        break
    time.sleep(OPENROUTER_RETRY_BACKOFF_SECONDS *
attempt)
        continue

    if response.ok:
        return response
    if (
        _is_retryable_status(response.status_code)
        and attempt < OPENROUTER_RETRY_ATTEMPTS
    ):
        time.sleep(OPENROUTER_RETRY_BACKOFF_SECONDS *
attempt)
            continue

    details = _response_error_message(response)
    msg = f"OpenRouter request failed
({response.status_code}): {details}"
    raise RuntimeError(msg)

    msg = f"OpenRouter request failed after retries:
{last_error!r}"
    raise RuntimeError(msg)

def get_llm_answer_service(prompt: str, context: str) ->
LlmAnswer:
    """Call OpenRouter chat completions and parse JSON from the
first choice."""
    api_key = _get_openrouter_api_key()
    headers = {
        "Authorization": f"Bearer {api_key}",
        "Content-Type": "application/json",
    }
    payload = _build_payload(prompt, context)
    response = _post_with_retry(headers, payload)
    data = response.json()
    raw_content = _extract_content(data)
    payload = _parse_json_payload(raw_content)
    model = str(data.get("model", OPENROUTER_MODEL))
    return LlmAnswer(model=model, raw_content=raw_content,
payload=payload)

async def build_vacancy_filter_allowed_values() -> dict[str,
list[str]]:
    """Build allowed filter values from unique vacancy fields in
PostgreSQL."""
    counts = await build_vacancy_filter_value_counts()
    return {key: sorted(counts[key]) for key in
VACANCY_FILTER_KEYS}

```

```

async def build_vacancy_filter_value_counts() -> dict[str,
dict[str, int]]:
    """Build ``value -> count`` maps for each filter field from
    PostgreSQL."""
    allowed_values = _empty_filter_count_object()
    async with async_session_factory() as session:
        for key in VACANCY_FILTER_KEYS:
            column = _FILTER_COLUMN_MAP[key]
            stmt = (
                select(column, func.count(column))
                .where(column.isnot(None))
                .group_by(column)
                .order_by(column)
            )
            result = await session.execute(stmt)
            value_counts: dict[str, int] = {}
            for value, count in result.all():
                normalized = _normalize_db_value(value)
                if normalized is not None:
                    value_counts[normalized] = int(count)
            allowed_values[key] = value_counts
    return allowed_values

async def get_vacancy_filters_from_text_async(user_message: str)
-> dict[str, Any]:
    """Build DB-based prompt, request filters from LLM, and
    validate response."""
    allowed_values = await build_vacancy_filter_allowed_values()
    prompt = _build_system_prompt(allowed_values)
    base_context = VACANCY_FILTERS_CONTEXT_TEMPLATE.format(
        user_message=user_message.strip()
    )
    context = base_context
    errors: list[str] = []
    for _ in range(LLM_VALIDATION_RETRY_ATTEMPTS):
        try:
            answer = get_llm_answer_service(prompt, context)
        except ValueError as error:
            errors = [str(error)]
        else:
            errors = _validate_llm_payload(answer.payload,
allowed_values)
            if not errors:
                return answer.payload
            context = _build_retry_context(base_context, errors)
    details = "; ".join(errors) if errors else "Unknown
validation error."
    raise RuntimeError(f"LLM response validation failed:
{details}")

def get_vacancy_filters_from_text(user_message: str) ->
dict[str, Any]:
    """Synchronous wrapper for LLM-based vacancy filter
    extraction."""
    try:
        asyncio.get_running_loop()
    except RuntimeError:

```

```

        return
    asyncio.run(get_vacancy_filters_from_text_async(user_message))
    msg = "Use get_vacancy_filters_from_text_async() inside
    async context."
    raise RuntimeError(msg)

```

Листинг А.3 – Модуль LLM-извлечения и валидации структурированных фильтров

Примечание – Источник: собственная разработка.

```

"""Orchestrate user search: LLM filter extraction then filtered
embedding search."""

from __future__ import annotations

import asyncio
from typing import Any

from common.constants.embedding import DEFAULT_SIMILARITY_TOP_K
from common.constants.llm import VACANCY_FILTER_KEYS
from common.constants.search import (
    DEFAULT_MAX_VALUES_PER_FILTER,
    DEFAULT_MIN_FILTERED_CANDIDATES,
    DOMAIN_BOOST_STEP,
    PROTECTED_RELAX_FILTER_KEYS,
    PROTECTED_RELAX_WEIGHT_PENALTY,
    ROLE_KEYWORDS_HARD_LOCK,
)
from sqlalchemy import Select, func, or_, select

from features.database.db import async_session_factory
from features.database.models import Vacancy
from features.embedding.schemas import SimilaritySearchResult
from features.embedding.similarity_search_service import
similarity_search
from features.llm.service import (
    build_vacancy_filter_value_counts,
    get_vacancy_filters_from_text_async,
)
from features.search.schemas import FilterRelaxStep,
UserSearchResult

FILTER_COLUMN_MAP = {
    "payment_frequency": Vacancy.payment_frequency,
    "experience": Vacancy.experience,
    "employment": Vacancy.employment,
    "hiring_format": Vacancy.hiring_format,
    "schedule": Vacancy.schedule,
    "hours": Vacancy.hours,
    "work_format": Vacancy.work_format,
}

DOMAIN_KEYWORDS = {
    "horeca": ("бариста", "кофе", "кофейн", "десерт",
    "кондитер", "пекар", "кухн"),
    "it": ("python", "backend", "developer", "разработ",
    "программист", "fastapi"),

```

```

        "logistics": ("склад", "логист", "комплектов", "кладов",
"грузчик"),
    }

def _empty_filters() -> dict[str, list[str]]:
    return {key: [] for key in VACANCY_FILTER_KEYS}

def _empty_weighted_filters() -> dict[str, list[dict[str, float
| str]]]:
    return {key: [] for key in VACANCY_FILTER_KEYS}

def _clone_non_empty(filters: dict[str, list[str]]) -> dict[str,
list[str]]:
    return {key: list(values) for key, values in filters.items()
if values}

def _empty_filter_counts() -> dict[str, dict[str, int]]:
    return {key: {} for key in VACANCY_FILTER_KEYS}

def _with_all_count_keys(
    counts: dict[str, dict[str, int]],
) -> dict[str, dict[str, int]]:
    completed = _empty_filter_counts()
    completed.update(counts)
    return completed

def _with_all_keys(filters: dict[str, list[str]]) -> dict[str,
list[str]]:
    completed = _empty_filters()
    completed.update(filters)
    return completed

def _normalize_weighted_filters(
    payload: dict[str, Any],
) -> dict[str, list[tuple[str, float]]]:
    normalized: dict[str, list[tuple[str, float]]] = {
        key: [] for key in VACANCY_FILTER_KEYS
    }
    for key in VACANCY_FILTER_KEYS:
        items = payload.get(key, [])
        if not isinstance(items, list):
            msg = f"Filter '{key}' must be a list of objects."
            raise ValueError(msg)
        weight_by_value: dict[str, float] = {}
        for item in items:
            if not isinstance(item, dict):
                continue
            raw_value = item.get("value")
            raw_weight = item.get("weight")
            if not isinstance(raw_value, str) or not
raw_value.strip():
                continue
            if not isinstance(raw_weight, int | float):

```

```

        continue
        value = raw_value.strip()
        weight = float(raw_weight)
        weight_by_value[value] =
max(weight_by_value.get(value, 0.0), weight)
        normalized[key] = sorted(
            ((value, weight) for value, weight in
weight_by_value.items()),
            key=lambda item: item[1],
            reverse=True,
        )[:DEFAULT_MAX_VALUES_PER_FILTER]
    return normalized

def _serialize_weighted_filters(
    filters: dict[str, list[tuple[str, float]]],
) -> dict[str, list[dict[str, float | str]]]:
    result = _empty_weighted_filters()
    for key in VACANCY_FILTER_KEYS:
        result[key] = [
            {"value": value, "weight": round(weight, 4)}
            for value, weight in filters[key]
        ]
    return result

def _plain_filters_from_weighted(
    filters: dict[str, list[tuple[str, float]]],
) -> dict[str, list[str]]:
    plain = _empty_filters()
    for key in VACANCY_FILTER_KEYS:
        plain[key] = [value for value, _ in filters[key]]
    return plain

def _apply_filters(
    stmt: Select[tuple[int]], filters: dict[str, list[str]]
) -> Select[tuple[int]]:
    for key, values in filters.items():
        if values:
            stmt =
stmt.where(_FILTER_COLUMN_MAP[key].in_(values))
    return stmt

def _normalize_role_keywords(payload: dict[str, Any]) ->
list[str]:
    role_keywords = payload.get("role_keywords", [])
    if not isinstance(role_keywords, list):
        return []
    normalized = sorted(
        {
            item.strip().lower()
            for item in role_keywords
            if isinstance(item, str) and item.strip()
        }
    )
    return normalized[:5]

```

```

def _apply_role_keywords(
    stmt: Select[tuple[int]], role_keywords: list[str]
) -> Select[tuple[int]]:
    if not ROLE_KEYWORDS_HARD_LOCK or not role_keywords:
        return stmt
    clauses = [
        or_(
            Vacancy.title.ilike(f"%{keyword}%"),
            Vacancy.skills.ilike(f"%{keyword}%"),
        )
        for keyword in role_keywords
    ]
    return stmt.where(or_(*clauses))

async def _count_filtered_candidates(
    filters: dict[str, list[str]],
    *,
    role_keywords: list[str],
) -> int:
    stmt = select(func.count(Vacancy.id))
    stmt = _apply_filters(stmt, filters)
    stmt = _apply_role_keywords(stmt, role_keywords)
    async with async_session_factory() as session:
        count = await session.scalar(stmt)
    return int(count or 0)

def _build_selected_value_counts(
    filters: dict[str, list[tuple[str, float]]],
    value_counts: dict[str, dict[str, int]],
) -> dict[str, dict[str, int]]:
    selected_counts = _empty_filter_counts()
    for key in VACANCY_FILTER_KEYS:
        for value, _ in filters.get(key, []):
            selected_counts[key][value] = value_counts.get(key,
{}).get(value, 0)
    return selected_counts

def _build_field_drop_order(
    weighted_filters: dict[str, list[tuple[str, float]]],
    protected_keys: set[str],
) -> list[tuple[str, float]]:
    fields: list[tuple[str, float]] = []
    for key in VACANCY_FILTER_KEYS:
        values = weighted_filters[key]
        if values:
            field_weight = max(weight for _, weight in values)
            fields.append((key, field_weight))

    def _effective_weight(item: tuple[str, float]) -> float:
        key, weight = item
        if key in protected_keys:
            return min(1.0, weight +
PROTECTED_RELAX_WEIGHT_PENALTY)
        return weight

    return sorted(
        fields,

```

```

        key=lambda item: (
            _effective_weight(item),
            item[0],
        ),
    )

async def _relax_filters(
    weighted_filters: dict[str, list[tuple[str, float]]],
    *,
    value_counts: dict[str, dict[str, int]],
    role_keywords: list[str],
    min_candidates: int,
) -> tuple[dict[str, list[str]], dict[str, list[str]],
list[FilterRelaxStep], int]:
    active_filters =
_clone_non_empty(_plain_filters_from_weighted(weighted_filters))
    weighted_by_key = {key: list(values) for key, values in
weighted_filters.items()}
    dropped_filters = empty_filters()
    relax_steps: list[FilterRelaxStep] = []
    candidate_count = await _count_filtered_candidates(
        active_filters, role_keywords=role_keywords
    )
    if candidate_count >= min_candidates:
        return (
            _with_all_keys(active_filters),
            dropped_filters,
            relax_steps,
            candidate_count,
        )

    protected_keys = set(PROTECTED_RELAX_FILTER_KEYS)
    drop_order = _build_field_drop_order(weighted_by_key,
protected_keys)
    for step_index, (key, weight) in enumerate(drop_order,
start=1):
        removed_values = [value for value, _ in
weighted_by_key.get(key, [])]
        if not removed_values:
            continue
        candidates_before = candidate_count
        dropped_filters[key].extend(removed_values)
        active_filters.pop(key, None)
        candidate_count = await _count_filtered_candidates(
            active_filters, role_keywords=role_keywords
        )
        removed_total_count = sum(
            value_counts.get(key, {}).get(value, 0) for value in
removed_values
        )
        relax_steps.append(
            FilterRelaxStep(
                step=step_index,
                filter_key=key,
                filter_value=" ".join(removed_values),
                removed_weight=weight,
                value_count_in_db=removed_total_count,
                candidates_before=candidates_before,
                candidates_after=candidate_count,
            )

```

```

        )
    )
    if candidate_count >= min_candidates:
        break
    return _with_all_keys(active_filters), dropped_filters,
    relax_steps, candidate_count

def _query_domain_keywords(query: str, role_keywords: list[str])
-> set[str]:
    query_lower = query.lower()
    matched: set[str] = set(role_keywords)
    for keywords in _DOMAIN_KEYWORDS.values():
        if any(keyword in query_lower for keyword in keywords):
            matched.update(keywords)
    return matched

def _rank_with_domain_boost(
    user_query: str,
    role_keywords: list[str],
    vacancies: list[SimilaritySearchResult],
) -> list[SimilaritySearchResult]:
    keywords = _query_domain_keywords(user_query, role_keywords)
    if not keywords:
        return vacancies
    scored: list[tuple[float, float, SimilaritySearchResult]] =
    []
    for vacancy in vacancies:
        haystack = f"{vacancy.title} {vacancy.skills}".lower()
        matches = sum(1 for keyword in keywords if keyword in
        haystack)
        boosted_distance = vacancy.cosine_distance -
        DOMAIN_BOOST_STEP * min(matches, 3)
        scored.append((boosted_distance,
        vacancy.cosine_distance, vacancy))
    scored.sort(key=lambda item: (item[0], item[1]))
    return [item[2] for item in scored]

async def user_search(
    user_query: str,
    *,
    limit: int = DEFAULT_SIMILARITY_TOP_K,
    min_candidates: int = DEFAULT_MIN_FILTERED_CANDIDATES,
) -> UserSearchResult:
    """
    Run user-facing search flow:
    1) infer structured filters via LLM,
    2) relax low-weight fields until enough candidates,
    3) run embedding similarity search within filtered vacancy
    subset.
    """
    stripped_query = user_query.strip()
    if not stripped_query:
        return UserSearchResult(
            user_query="",
            role_keywords=[],
            llm_filters=_empty_weighted_filters(),
            all_value_counts=_empty_filter_counts(),

```

```

        selected_value_counts=_empty_filter_counts(),
        applied_filters=_empty_filters(),
        dropped_filters=_empty_filters(),
        relax_steps=[],
        candidate_count=0,
        vacancies=[],
    )
    raw_filters = await
get_vacancy_filters_from_text_async(stripped_query)
    role_keywords = _normalize_role_keywords(raw_filters)
    llm_weighted_filters =
_normalize_weighted_filters(raw_filters)
    value_counts = await build_vacancy_filter_value_counts()
    selected_value_counts = _build_selected_value_counts(
        llm_weighted_filters, value_counts
    )
    (
        applied_filters,
        dropped_filters,
        relax_steps,
        candidate_count,
    ) = await _relax_filters(
        llm_weighted_filters,
        value_counts=value_counts,
        role_keywords=role_keywords,
        min_candidates=min_candidates,
    )
    vacancies = await similarity_search(
        stripped_query,
        limit=limit,
        filters=applied_filters,
        role_keywords=role_keywords,
    )
    ranked_vacancies = _rank_with_domain_boost(stripped_query,
role_keywords, vacancies)
    return UserSearchResult(
        user_query=stripped_query,
        role_keywords=role_keywords,
        llm_filters=_serialize_weighted_filters(llm_weighted_filters),
        all_value_counts=_with_all_count_keys(value_counts),
        selected_value_counts=selected_value_counts,
        applied_filters=applied_filters,
        dropped_filters=dropped_filters,
        relax_steps=relax_steps,
        candidate_count=candidate_count,
        vacancies=ranked_vacancies,
    )

def run_user_search(
    user_query: str,
    *,
    limit: int = DEFAULT_SIMILARITY_TOP_K,
    min_candidates: int = DEFAULT_MIN_FILTERED_CANDIDATES,
) -> UserSearchResult:
    """Synchronous wrapper for :func:`user_search` (CLI scripts,
REPL)."""
    return asyncio.run(

```

```
        user_search(user_query, limit=limit,  
min_candidates=min_candidates)  
    )
```

Листинг А.4 – Модуль пользовательского поиска, релаксации и доменного
перанжирования

Примечание – Источник: собственная разработка.

ПРИЛОЖЕНИЕ Б

Набор бенчмарк-запросов и результаты прогона

В приложении представлен набор тестовых запросов, использованный для локальной оценки качества пользовательского поиска вакансий. Набор сформирован по материалам модуля backend/src/features/search/evaluation.py и включает 16 естественно-языковых запросов, отражающих типовые сценарии поиска работы молодыми специалистами и соискателями без опыта.

Локальный запуск выполнен 26.04.2026 в локальной среде проекта.

Таблица Б.1 – Результаты локального тестирования тестовых запросов

№	Запрос (сокращенно)	Ожидаемые ключевые слова	hit@1	hit@5	Заголовок вакансии в позиции top 1
1	Студент без опыта, ПВЗ Wildberries или Ozon, офлайн	пвз, пункт выдачи, wildberries, ozon	да	да	Менеджер пункта выдачи заказов Wildberries (Лиможа)
2	Стажировка или junior в интернет-маркетинге	маркетолог, интернет-маркетолога, digital	нет	нет	Junior Sales Manager
3	Бариста или работа в кофейне без опыта	бариста, кофе, кофейн	да	да	Бариста «Джезва»
4	Кладовщик 2/2, вечерние или ночные смены	кладовщик, склад	да	да	Кладовщик
5	Комплектовщик или сортировщик на складе	комплектовщик, сортировщик, склад	да	да	Комплектовщик товаров (д. Прилесье)

6	Менеджер по логистике, экспедитор или логист	логист, экспедитор	да	да	Менеджер по логистике
7	Подработка продавцом, частичная занятость	продавец	да	да	Продавец в магазин «Гренка»
8	Удаленная работа support или help desk specialist	support, help desk, специалист	да	да	Support Specialist (Help Desk Analyst)
9	Программист 1С, удаленно, полная занятость	программист 1с, 1с	да	да	Программист 1С
10	Официант или повар без опыта	официант, повар	да	да	Сотрудник ресторана KFC/ Официант/ Кассир
11	Водитель категории В, доставка	водитель	да	да	Водитель категории В
12	Стартовая роль в подборе персонала, IT, стажировка или junior	подбор, hr, рекрутер, персонал	нет	нет	Intern/Junior Salesforce Developer
13	Удаленный ассистент интернет-маркетолога	ассистент интернет-маркетолога, маркетолог	да	да	Ассистент интернет-маркетолога
14	Мерчендайзер или торговый представитель	мерчендайзер, торгов	да	да	Торговый представитель отдела продаж

15	Кондитер или помощник кондитера	кондитер, десерт	да	да	Кондитер- оформитель тортов, Гродно
16	ПВЗ Ozon или Wildberries, выдача посылок	пвз, ozon, wildberries, пункт выдачи	да	да	Продавец- кассир в пункт выдачи заказов, город Гродно

Окончание таблицы Б.1

Примечание – Источник: собственная разработка.

По итогам локального запуска показатель $hit@1$ составил 14 из 16, а показатель $hit@5$ составил 14 из 16. Полученные результаты свидетельствуют о приемлемом качестве поиска по большинству массовых пользовательских сценариев, однако устойчивость ранжирования для части специализированных запросов остается недостаточной.

Наибольшие затруднения зафиксированы для запросов, связанных с интернет-маркетингом, junior-вакансиями и подбором персонала.